

# HTML5

## Application Development

Student Edition

Presented By  
**LK LearnKey®**

# HTML5 Application Development Project Workbook

First Edition

LearnKey creates signature multimedia courseware. LearnKey provides expert instruction for popular computer software, technical certifications, and application development with dynamic video-based courseware and effective learning management systems. For a complete list of courses, visit <https://www.learnkey.com>.

All rights reserved. Unauthorized reproduction or distribution is prohibited.

# Table of Contents

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Introduction</b>                             | <b>1</b>  |
| Best Practices Using LearnKey's Online Training | 2         |
| Using This Workbook                             | 3         |
| Skills Assessment                               | 4         |
| HTML5 Application Development Video Times       | 5         |
| <b>Domain 1 Lesson 1</b>                        | <b>6</b>  |
| Fill-in-the-Blanks                              | 7         |
| Application Lifecycle Steps                     | 8         |
| Debug and Test Web Applications                 | 9         |
| <b>Domain 2 Lesson 1</b>                        | <b>10</b> |
| Fill-in-the-Blanks                              | 11        |
| Canvas: Shape                                   | 12        |
| Canvas: Color                                   | 13        |
| Canvas: Line                                    | 14        |
| <b>Domain 2 Lesson 2</b>                        | <b>15</b> |
| Fill-in-the-Blanks                              | 16        |
| Canvas: Translation                             | 17        |
| Canvas: Rotate                                  | 18        |
| Canvas: Scale                                   | 19        |
| Canvas: Interactivity                           | 20        |
| <b>Domain 2 Lesson 3</b>                        | <b>21</b> |
| Fill-in-the-Blanks                              | 22        |
| Scalable Vector Graphics (SVGs)                 | 23        |
| SVG: Shapes and Color                           | 24        |
| SVG: Filter Effects                             | 25        |
| <b>Domain 2 Lesson 4</b>                        | <b>26</b> |
| Fill-in-the-Blanks                              | 27        |
| Rounded Corners, Shadows, and Transparency      | 28        |
| Background Gradients                            | 29        |
| Typography and Web Open Font Format             | 30        |
| 2D and 3D Transformations                       | 31        |
| Keyframes                                       | 32        |

|                                    |           |
|------------------------------------|-----------|
| <b>Domain 2 Lesson 5</b>           | <b>33</b> |
| Fill-in-the-Blanks                 | 34        |
| CSS Image Filters                  | 35        |
| CSS Drop Shadow                    | 36        |
| <b>Domain 3 Lesson 1</b>           | <b>37</b> |
| Fill-in-the-Blanks                 | 38        |
| Form Elements                      | 39        |
| Input Validation                   | 40        |
| <b>Domain 4 Lesson 1</b>           | <b>41</b> |
| Fill-in-the-Blanks                 | 42        |
| Content Flow                       | 43        |
| Positioning                        | 44        |
| Content Overflow and CSS Reset     | 45        |
| <b>Domain 4 Lesson 2</b>           | <b>46</b> |
| Fill-in-the-Blanks                 | 47        |
| Grid View                          | 48        |
| Background Size                    | 49        |
| Images                             | 50        |
| Viewport                           | 51        |
| <b>Domain 4 Lesson 3</b>           | <b>52</b> |
| Fill-in-the-Blanks                 | 53        |
| Responsive Width and Media Queries | 54        |
| CSS Flexbox                        | 55        |
| CSS Flex Items                     | 56        |
| <b>Domain 4 Lesson 4</b>           | <b>57</b> |
| Fill-in-the-Blanks                 | 58        |
| Grid Container                     | 59        |
| Grid Items                         | 60        |
| Grid Templates                     | 61        |
| Grid Gaps and Lines                | 62        |
| <b>Domain 5 Lesson 1</b>           | <b>63</b> |
| Fill-in-the-Blanks                 | 64        |
| Instantiation                      | 65        |
| Properties                         | 66        |

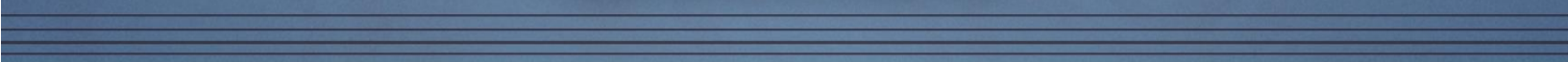
|                                                  |            |
|--------------------------------------------------|------------|
| Methods                                          | 67         |
| Inheritance                                      | 68         |
| <b>Domain 5 Lesson 2</b>                         | <b>69</b>  |
| Fill-in-the-Blanks                               | 70         |
| Sending and Receiving Data                       | 71         |
| XML                                              | 72         |
| <b>Domain 5 Lesson 3</b>                         | <b>73</b>  |
| Fill-in-the-Blanks                               | 74         |
| JSON                                             | 75         |
| Complex Objects                                  | 76         |
| Load and Save Files                              | 77         |
| <b>Domain 5 Lesson 4</b>                         | <b>78</b>  |
| Fill-in-the-Blanks                               | 79         |
| Gesture Events                                   | 80         |
| Multiple Events                                  | 81         |
| Event Object                                     | 82         |
| <b>Domain 5 Lesson 5</b>                         | <b>83</b>  |
| Fill-in-the-Blanks                               | 84         |
| Google Charts                                    | 85         |
| jQuery                                           | 86         |
| Geolocation                                      | 87         |
| <b>Domain 5 Lesson 6</b>                         | <b>88</b>  |
| Fill-in-the-Blanks                               | 89         |
| Local and Session Storage                        | 90         |
| <b>Appendix</b>                                  | <b>91</b>  |
| Glossary                                         | 92         |
| Objectives                                       | 100        |
| <b>HTML5 Application Development Lesson Plan</b> | <b>101</b> |
| Domain 1 Lesson Plan                             | 102        |
| Domain 2 Lesson Plan                             | 103        |
| Domain 3 Lesson Plan                             | 105        |
| Domain 4 Lesson Plan                             | 106        |
| Domain 5 Lesson Plan                             | 107        |



A series of thin, light blue lines resembling circuit traces or a stylized border. They start from the left edge, run horizontally, and then branch out into various geometric shapes like L-shapes and small circles, extending towards the right edge.

# Introduction

**HTML5**  
Application Development

A series of thin, light blue lines resembling circuit traces or a stylized border. They start from the left edge, run horizontally, and then branch out into various geometric shapes like L-shapes and small circles, extending towards the right edge.

# Best Practices Using LearnKey's Online Training

LearnKey offers video-based training solutions that are flexible enough to accommodate private students and educational facilities and organizations.

Our course content is presented by top experts in their respective fields and provides clear and comprehensive information. The full line of LearnKey products has been extensively reviewed to meet superior standards of quality. Our course content has also been endorsed by organizations such as Certiport, CompTIA®, Cisco, and Microsoft. However, it is the testimonials given by countless satisfied customers that truly set us apart as leaders in the information training world.

LearnKey experts are highly qualified professionals who offer years of job and project experience in their subjects. Each expert has been certified at the highest level available for their field of expertise. This expertise provides the student with the knowledge necessary to obtain top-level certifications in their chosen field.

Our accomplished instructors have a rich understanding of the content they present. Effective teaching encompasses presenting the basic principles of a subject and understanding and appreciating organization, real-world application, and links to other related disciplines. Each instructor represents the collective wisdom of their field and within our industry.

## Our Instructional Technology

Each course is independently created based on the manufacturer's standard objectives for which the course was developed.

We ensure that the subject matter is up-to-date and relevant. We examine the needs of each student and create training that is both interesting and effective. LearnKey training provides auditory, visual, and kinesthetic learning materials to fit diverse learning styles.

## Course Training Model

The course training model allows students to undergo basic training, building upon primary knowledge and concepts to more advanced application and implementation. In this method, students will use the following toolset:

**Pre-assessment:** The pre-assessment is used to determine the student's prior knowledge of the subject matter. It will also identify a student's strengths and weaknesses, allowing them to focus on the specific subject matter they need to improve the most. Students should not necessarily expect a passing score on the pre-assessment as it is a test of prior knowledge.

**Video training sessions:** Each training course is divided into sessions or domains and lessons with topics and subtopics. LearnKey recommends incorporating all available external resources into your training, such as student workbooks, glossaries, course support files, and additional customized instructional material. These resources are located in the folder icon at the top of the page.

**Exercise labs:** Labs are interactive activities that simulate situations presented in the training videos. Step-by-step instructions and live demonstrations are provided.

**Post-assessment:** The post-assessment is used to determine the student's knowledge gained from interacting with the training. In taking the post-assessment, students should not consult the training or any other materials. A passing score is 80 percent or higher. If the individual does not pass the post-assessment the first time, LearnKey recommends incorporating external resources, such as the workbook and additional customized instructional material.

**Workbook:** The workbook has various activities, such as fill-in-the-blank worksheets, short answer questions, practice exam questions, and group and individual projects that allow the student to study and apply concepts presented in the training videos.

# Using This Workbook

This project workbook contains practice projects and exercises to reinforce the knowledge you have gained through the video portion of the **HTML5 Application Development** course. The purpose of this workbook is twofold. First, get you further prepared to pass the HTML5 Application Development exam, and second, to get you job-ready skills and increase your employability in the area of web development.

The projects within this workbook follow the order of the video portion of this course. To save your answers in this workbook, you must first download a copy to your computer. You will not be able to save your answers in the web version. You can complete the workbook exercises as you go through each section of the course, complete several of these at the end of each domain, or complete them after viewing the entire course. The key is to go through these projects to strengthen your knowledge of this course.

Each project is based upon a specific video (or videos) in the course and specific test objectives. The materials you will need for this course include:

- LearnKey's **HTML5 Application Development** courseware.
- The course project files. All applicable project files are located in the support area where you downloaded this workbook.
- An Integrated Development Environment (IDE) or code editing application such as Visual Studio Code.

## For Teachers

LearnKey is proud to provide extra support to instructors upon request. For your benefit as an instructor, we also provide an instructor support .zip file containing answer keys, completed versions of the workbook project files, and other teacher resources. This .zip file is available within your learning platform's admin portal.

## Notes

- Extra teacher notes, when applicable, are in the Project Details box within each exercise.
- Exam objectives are aligned with the course objectives listed in each project, and project file names correspond with these numbers.
- The Finished folder in each domain has reference versions of each project. These can help you grade projects.
- Short answers may vary but should be similar to those provided in this workbook.
- Teachers may consider asking students to add their initials, student ID, or other personal identifiers at the end of each saved project.
- Refer to your course representatives for further support.

We value your feedback about our courses. If you have any questions, comments, or concerns, please let us know by visiting <https://about.learnkey.com>.



# Skills Assessment

**Instructions:** Rate your skills on the following tasks from 1-5 (1 being needs improvement, 5 being excellent).

| Skills                                                                     | 1 | 2 | 3 | 4 | 5 |
|----------------------------------------------------------------------------|---|---|---|---|---|
| Describe the application lifecycle management stages.                      |   |   |   |   |   |
| Debug and test web apps.                                                   |   |   |   |   |   |
| Use the canvas element to create graphics and animations.                  |   |   |   |   |   |
| Use the SVG element to create and display graphics.                        |   |   |   |   |   |
| Transform, style, and enhance text and graphics.                           |   |   |   |   |   |
| Apply CSS filters to images.                                               |   |   |   |   |   |
| Construct and analyze markup that uses form elements.                      |   |   |   |   |   |
| Configure input validation.                                                |   |   |   |   |   |
| Manage content layout, positioning, and flow using CSS.                    |   |   |   |   |   |
| Construct layouts using responsive design.                                 |   |   |   |   |   |
| Construct flexible, responsive layouts using CSS flexbox.                  |   |   |   |   |   |
| Construct grid-based layouts using CSS grid.                               |   |   |   |   |   |
| Create and use custom classes.                                             |   |   |   |   |   |
| Perform data access using JavaScript.                                      |   |   |   |   |   |
| Construct code that responds to events using event listeners and handlers. |   |   |   |   |   |
| Construct code that uses JavaScript APIs.                                  |   |   |   |   |   |
| Manage the state of an application.                                        |   |   |   |   |   |
|                                                                            |   |   |   |   |   |
|                                                                            |   |   |   |   |   |

# HTML5 Application Development Video Times

| Domain 1                         | Video Time     |
|----------------------------------|----------------|
| Application Lifecycle Management | 0:04:45        |
| Debug and Test Web Apps          | 0:09:06        |
| <b>Total Time</b>                | <b>0:13:51</b> |

| Domain 2              | Video Time     |
|-----------------------|----------------|
| Canvas Element        | 0:16:23        |
| SVG Element           | 0:09:32        |
| Graphics Effects      | 0:13:31        |
| CSS Filters on Images | 0:07:37        |
| <b>Total Time</b>     | <b>0:47:03</b> |

| Domain 3                   | Video Time     |
|----------------------------|----------------|
| Form Markup                | 0:03:46        |
| Configure Input Validation | 0:08:08        |
| <b>Total Time</b>          | <b>0:11:54</b> |

| Domain 4                      | Video Time     |
|-------------------------------|----------------|
| Layout, Positioning, and Flow | 0:11:55        |
| Responsive Design             | 0:13:55        |
| Flexible Responsive Layouts   | 0:06:36        |
| Grid-Based Layouts            | 0:08:33        |
| <b>Total Time</b>             | <b>0:40:59</b> |

| Domain 5                       | Video Time     |
|--------------------------------|----------------|
| Custom Classes                 | 0:13:04        |
| Data Access Through JavaScript | 0:22:36        |
| Event Listeners and Handlers   | 0:13:23        |
| JavaScript APIs                | 0:14:17        |
| Application States             | 0:05:34        |
| <b>Total Time</b>              | <b>1:08:54</b> |

# Domain 1

# Lesson 1

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 1 Lesson 1, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. A \_\_\_\_\_ is a development environment that mimics an existing system environment. [Application Lifecycle Steps]
2. Regular expressions can be used with the HTML \_\_\_\_\_ attribute or within JavaScript. [Input Validation Errors]
3. Runtime errors occur during the period of code \_\_\_\_\_. [Runtime Errors]
4. Breakpoints allow developers to \_\_\_\_\_ code by stepping through it line by line. [Breakpoints]

# Application Lifecycle Steps

This lesson covers the six-step plan for managing an application throughout its lifecycle. These steps cover everything from the pre-planning of an idea to the steps taken post-deployment to ensure that an application continues to run as expected long after its initial deployment.

## Purpose

Upon completing this project, you will better understand the six steps that make up the application lifecycle.

## Steps for Completion

1. Match the lifecycle step with its description.

|           |            |             |
|-----------|------------|-------------|
| A. Plan   | C. Develop | E. Deploy   |
| B. Design | D. Test    | F. Maintain |

- \_\_\_\_\_ Teams may use Git and GitHub to collaborate and track changes for an application
- \_\_\_\_\_ Pre-determined intervals to assess software for functionality and security
- \_\_\_\_\_ Big picture phase that considers how an application will fit into an existing environment
- \_\_\_\_\_ Often performed in a sandbox environment, protecting current systems
- \_\_\_\_\_ Application is made available to all appropriate parties across all appropriate networks
- \_\_\_\_\_ Decisions are made about file structure, program architecture, and user interface

## Project Details

### Project file

N/A

### Estimated completion time

10 minutes

### Video reference

#### Domain 1

**Topic:** Application Lifecycle Management

**Subtopic:** Application Lifecycle Steps

### Objectives covered

**1** Application Lifecycle Management

**1.1** Describe the application lifecycle management stages

**1.1.1** Plan, design, develop, test, deploy, and maintain



# Debug and Test Web Applications

The process of debugging and testing web applications continues throughout the lifecycle of any application. Validating user input is one way to avoid crashes and security threats. Runtime errors can crash an application, so they should be avoided at all costs. Lastly, breakpoints allow developers to walk through the code line by line and analyze individual parts of an application.

## Purpose

Upon completing this project, you will better understand input validation, runtime errors, and breakpoints as they relate to debugging and testing web applications.

## Steps for Completion

1. List three HTML attributes used to validate user input.
  - a. \_\_\_\_\_
  - b. \_\_\_\_\_
  - c. \_\_\_\_\_
2. Regular expressions are used to test for a \_\_\_\_\_ or sequence of characters.
3. Webpages are best previewed if they are published to a \_\_\_\_\_.
4. Runtime errors typically send a message to the console, found in the Developer Tools by selecting \_\_\_\_\_ on the keyboard.
5. Describe the two ways developers can add breakpoints to their code.

---



---

### Project Details

#### Project file

N/A

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 1

**Topic:** Debug and Test Web Apps

**Subtopic:** Input Validation Errors;  
Runtime Errors; Breakpoints

#### Objectives covered

**1** Application Lifecycle Management

**1.2** Debug and test web apps

**1.2.1** Input validation errors

**1.2.2** Runtime errors

**1.2.3** Breakpoints

# Domain 2

# Lesson 1

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 2 Lesson 1, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. The \_\_\_\_\_ method draws a rectangle that is filled. [Shape]
2. To draw a circle, the arc method takes five parameters. x-coordinate, y-coordinate, \_\_\_\_\_, startingAngle and endingAngle. [Shape]
3. The strokeStyle and fillStyle properties can be set to \_\_\_\_\_ values, hex color codes, or common color names. [Color]
4. A line is not drawn to the canvas until the \_\_\_\_\_ method is called. [Line]

# Canvas: Shape

The canvas is an HTML element used to create graphic elements and animations using JavaScript code. This lesson will focus on creating basic shapes on a canvas. For the exam, know the correct syntax for creating a canvas element and know the basic shapes that can be drawn to a canvas.

## Purpose

Upon completing this project, you will better understand how to draw squares, rectangles, circles, and lines to an HTML canvas.

## Steps for Completion

1. Open **index.html** and **canvas.js** from the Domain 2 Student folder.
2. On line 10, add a line of code to draw a rectangle starting at (0, 100). The rectangle should measure 100px wide and 200px tall.
3. On line 17, add a line of code to draw a circle at the coordinates (400, 200). The circle radius should measure 50px.
4. Take a screenshot of the **canvas.js** file that includes lines 10 and 17.
5. Save the screenshot as **211-Completed**.

## Project Details

### Project file

index.html  
canvas.js

### Estimated completion time

15 minutes

### Video reference

#### Domain 2

**Topic:** Canvas Element

**Subtopic:** Shape

### Objectives covered

**2** Graphics and Animation

**2.1** Use the canvas element to create graphics and animations

**2.1.1** Shape

# Canvas: Color

Color can be set using RGB values, hex color codes, and common color names. RGB values and hex colors are recommended due to their high level of specificity versus simple color names. The color is first specified in the `strokeStyle` or `fillStyle` property of the canvas element to color canvas elements. Then the `stroke()` method or `fill()` method adds color to the element.

## Purpose

Upon completing this project, you will better understand how to specify colors for canvas elements.

## Steps for Completion

1. Open **index.html** and **canvas.js** from the Domain 2 Student folder.
2. On line 8, add a line of code that sets the stroke color to #F5F5F5.
3. On line 22, add a line of code that sets the fill color to #F5F5F5.
4. Take a screenshot of the **canvas.js** file that includes lines 8 and 22.
5. Save the screenshot as **212-Completed**.

## Project Details

### Project file

index.html  
canvas.js

### Estimated completion time

10 minutes

### Video reference

#### Domain 2

**Topic:** Canvas Element

**Subtopic:** Color

### Objectives covered

**2** Graphics and Animation

**2.1** Use the canvas element to create graphics and animations

**2.1.2** Color



# Canvas: Line

Lines are drawn to a canvas using a combination of methods. First, the path must begin and position itself on a canvas. Then, after outlining another coordinate on the canvas, the stroke method can draw a line. For the exam, be familiar with the basic methods used to draw a line to a canvas.

## Purpose

Upon completing this project, you will better understand how to draw straight lines to a canvas.

## Steps for Completion

1. Open **index.html** and **canvas.js** from the Domain 2 Student folder.
2. On line 28, position the start of the line at the coordinates (400, 0).
3. On line 29, draw a line to the coordinates (400, 400).
4. Take a screenshot of the **canvas.js** file that includes lines 28 and 29.
5. Save the screenshot as **213-Completed**.

### Project Details

#### Project file

index.html  
canvas.js

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 2

**Topic:** Canvas Element

**Subtopic:** Line

#### Objectives covered

**2** Graphics and Animation

**2.1** Use the canvas element to create graphics and animations

**2.1.3** Line

# Domain 2

# Lesson 2

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 2 Lesson 2, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. The translate method only changes the frame of reference for \_\_\_\_\_ objects. [Translation]
2. The rotate method takes just one parameter, an angle value in \_\_\_\_\_. [Rotate]
3. The scale method affects the size and \_\_\_\_\_ of an element. [Scale]
4. The offsetX and offsetY properties of the Click Event or Pointer Event are based on the pointer's relation to the \_\_\_\_\_ element. [Interactivity]

# Canvas: Translation

The translate method is used to specify a new frame of reference for canvas methods. The coordinates specified in the translate method are then considered (0, 0) for the remainder of the canvas elements. You can visualize this method by thinking of an artist at a drawing table. Without moving their drawing hand, the canvas is shifted beneath them, changing the frame of reference. This method affects only the canvas drawings made after the method is executed.

## Purpose

Upon completing this project, you will better understand how the translate method affects canvas elements.

## Steps for Completion

1. Open **index.html** and **canvas1.js** from the Domain 2 Student folder.
2. On the **index.html** page, ensure that the **canvas1.js** file is linked on line 15. Also, ensure that the link to **canvas.js** is commented out on line 14.
3. On line 7 of **canvas1.js**, add a line of code translates the point of reference for the canvas to the coordinates (250, 250).
4. Take a screenshot of lines 6 through 8 of **canvas1.js**.
5. Save the screenshot as **214-Completed**.

### Project Details

#### Project file

index.html  
canvas1.js

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Canvas Element

**Subtopic:** Translation

#### Objectives covered

**2** Graphics and Animation

**2.1** Use the canvas element to create graphics and animations

**2.1.4** Translation/move

# Canvas: Rotate

The rotate method is used to rotate the canvas around the point (0, 0). Positive values rotate the canvas clockwise while negative numbers rotate the canvas counterclockwise. Like the translate method, the rotate method does not alter how previous elements have been drawn to a canvas.

## Purpose

Upon completing this project, you will better understand how to add the rotate method to a canvas.

## Steps for Completion

1. Open **index.html** and **canvas1.js** from the Domain 2 Student folder.
2. Comment out the translate example on lines 6 through 8.
3. Uncomment the rotate example on lines 11-17
4. On line 14, add code to rotate the canvas and draw the second rectangle with a 45-degree clockwise rotation.
5. Take a screenshot of lines 11-17 of **canvas1.js**.
6. Save the screenshot as **215-Completed**.

## Project Details

### Project file

index.html  
canvas1.js

### Estimated completion time

5 minutes

### Video reference

#### Domain 2

**Topic:** Canvas Element

**Subtopic:** Rotate

### Objectives covered

**2** Graphics and Animation

**2.1** Use the canvas element to create graphics and animations

**2.1.5** Rotate

# Canvas: Scale

The scale method is used to adjust the positioning and size of shapes on a canvas. Like the rotate and translation methods, the scale method will only affect shapes drawn after the method is called.

## Purpose

Upon completing this project, you will better understand how to use the scale method on canvas elements.

## Steps for Completion

1. Open **index.html** and **canvas1.js** from the Domain 2 Student folder.
2. Comment out the rotate example on lines 11-17.
3. Uncomment the scale example on lines 20-23.
4. On line 21, add code that positions the circle on line 22 at the coordinates (200, 200) and makes the circle's radius twice that of the circle on line 20.
5. Take a screenshot of lines 20-23 of **canvas1.js**.
6. Save the screenshot as **216-Completed**.

### Project Details

#### Project file

index.html  
canvas1.js

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 2

**Topic:** Canvas Element

**Subtopic:** Scale

#### Objectives covered

**2** Graphics and Animation

**2.1** Use the canvas element to create graphics and animations

**2.1.6** Scale



# Canvas: Interactivity

With canvas elements drawn using JavaScript, there are many ways to make these elements dynamic and interactive. This project focuses on using JavaScript code to gather user input and alter canvas elements dynamically. Later projects will focus on event listeners and animations.

## Purpose

Upon completing this project, you will better understand the relationship between JavaScript and the canvas and how that relationship is used to create dynamic web content.

## Steps for Completion

1. Open **index.html** and **coords.js** from the Domain 2 Student folder.
2. Verify that the **coords.js** file is linked to the **index.html** page on line 16 of **index.html**.
3. In the **coords.js** file on line 9, call the `drawCircle` function and pass the `offsetX` and `offsetY` values from the event object as parameters.
4. Take a screenshot of lines 4 through 10 of **coords.js**.
5. Save the screenshot as **217a-Completed**.
6. Preview the **index.html** page. Select inside of the canvas. Open the console and inspect the event object.
7. Take a screenshot of the console with the event object.
8. Save the screenshot as **217b-Completed**.

## Project Details

### Project file

index.html  
coords.js

### Estimated completion time

10 minutes

### Video reference

#### Domain 2

**Topic:** Canvas Element

**Subtopic:** Interactivity

### Objectives covered

**2** Graphics and Animation

**2.1** Use the canvas element to create graphics and animations

**2.1.7** Interactivity

# Domain 2

# Lesson 3

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 2 Lesson 3, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Canvases are drawn using JavaScript, while Scalable Vector Graphics (SVGs) are drawn using \_\_\_\_\_. [Advantages]
2. Referenced XML is referring to XML that is not directly coded in the \_\_\_\_\_ for a webpage. [Inline vs. Referenced XML]
3. SVG shapes can include rectangles, circles, ellipses, lines, polylines, polygons, and \_\_\_\_\_. [Shapes]
4. SVG rectangles and circles accept a color value from the \_\_\_\_\_ attribute. [SVG Color]
5. SVG filter effects are added without using \_\_\_\_\_, which is typically used to style HTML elements on a webpage. [SVG Filter Effects]

# Scalable Vector Graphics (SVGs)

SVGs are two-dimensional graphics based on XML-style markup. SVGs are best known for their ability to be resized with no pixelation since the browser draws the image. Using XML markup to render images makes it easier for developers to create flexible graphics that look optimal on all screen sizes.

## Purpose

Upon completing this project, you will better understand SVGs and what purpose they serve in web development.

## Steps for Completion

1. Open **about.html** from the Domain 2 Student folder.
2. On line 98, add a line of code that references the `baseball.svg` file in the `images` folder. The image should be 700px wide. The image should also be assigned to the `baseball-img` class.
3. Take a screenshot of the **about.html** file that includes line 98.
4. Save the screenshot as **222-Completed**.

### Project Details

#### Project file

about.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 2

**Topic:** SVG Element

**Subtopic:** Advantages; Inline vs. Referenced XML

#### Objectives covered

**2** Graphics and Animation

**2.2** Use the SVG element to create and display graphics

**2.2.1** Advantages

**2.2.2** Inline vs. referenced XML

# SVG: Shapes and Color

Basic SVG shapes include rectangles, circles, ellipses, lines, polylines, polygons, and paths. All shapes do not accept the same parameters. While it is not important to memorize the parameters for each shape, it is important to recognize common errors in SVG code.

## Purpose

Upon completing this project, you will better understand how to draw basic SVG shapes.

## Steps for Completion

1. Open **about1.html** from the Domain 2 Student folder.
2. On line 38, add a line of code to draw a rectangle starting at the x and y-coordinates of (100, 25). The rx and ry values should be 3. The width and height of the rectangle should be 100 by 250. The fill value should be #505050. Preview the page. The SVG should resemble a stoplight.
3. Using the color value from line 41, fill the circle on line 42. Using the color value from line 43, fill the circle on line 44. Preview the page. There should be three colors for each stoplight circle.
4. Uncomment the code on line 47 to add a single line across the top of the SVG.
5. Take a screenshot of lines 37-48 of **about1.html**.
6. Save the screenshot as **224-Completed**.

### Project Details

#### Project file

about1.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 2

**Topic:** SVG Element

**Subtopic:** Shapes; SVG Color

#### Objectives covered

**2** Graphics and Animation

**2.2** Use the SVG element to create and display graphics

**2.2.3** Shapes

**2.2.4** Color

# SVG: Filter Effects

SVG filter effects are implemented by defining a filter and then linking that filter to an SVG element. SVG filters can be added inline as well. However, using a filter reference makes it much easier to reuse a specific filter. This project will focus on the blur effect, but this same technique can be used to produce drop shadows, linear transitions, and radial transitions, as well as other filter effects.

## Purpose

Upon completing this project, you will better understand how to use SVG filter effects to style SVGs.

## Steps for Completion

1. Open **about.html** from the Domain 2 Student folder.
2. Preview **about.html**. Note the clear solid black line which outlines the sun in the SVG. Return to the code editor.
3. On line 52, add a reference to the blur filter on line 48. Preview the page again. The outer border of the sun should be blurred.
4. Take a screenshot of lines 47-52 of **about.html**.
5. Save the screenshot as **225-Completed**.

### Project Details

#### Project file

about.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 2

**Topic:** SVG Element

**Subtopic:** SVG Filter Effects

#### Objectives covered

**2** Graphics and Animation

**2.2** Use the SVG element to create and display graphics

**2.2.5** SVG filter effects



# Domain 2

# Lesson 4

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 2 Lesson 4, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. The \_\_\_\_\_ property adds rounded corners to an element. [Rounded Corners, Shadows, and Transparency]
2. The \_\_\_\_\_ parameter can be added to a linear gradient to change the direction of the color transition from vertical to horizontal. [Background Gradients]
3. The \_\_\_\_\_ property is used to specify whether text should be italicized. [Typography and WOFF]
4. The \_\_\_\_\_ property is used to move an element from one position to another. [2D and 3D Transformations]
5. The \_\_\_\_\_ property is used to specify that an animation should loop infinitely. [Keyframes]

# Rounded Corners, Shadows, and Transparency

Styling text and graphics involves specifying CSS properties in a stylesheet. This project will focus on applying the border-radius property, box-shadow property, and opacity property to an image. For the exam, know the effect that each of these properties has and be able to identify them among other CSS properties.

## Purpose

Upon completing this project, you will better understand how to style text and graphics using the border-radius, box-shadow, and opacity properties.

## Steps for Completion

1. Open **about.html** and **main.css** from the Domain 2 Student folder.
2. Find the image-paragraph tag on line 219 of **main.css**.
3. On line 220, add a line of code to round the image's borders by five pixels.
4. On line 221, add a line of code to create a shadow, five pixels below and five pixels to the right of the image. Set the blur and spread for the shadow to ten pixels. Lastly, set the color of the shadow to grey.
5. On line 222, add a line of code to make the image 50% transparent.
6. Take a screenshot of the image-paragraph class on lines 219-225 on **main.css**.
7. Save the screenshot as **231a-Completed**.

## Project Details

### Project file

about.html  
main.css

### Estimated completion time

10 minutes

### Video reference

#### Domain 2

**Topic:** Graphics Effects

**Subtopic:** Rounded Corners, Shadows, and Transparency

### Objectives covered

#### 2 Graphics and Animation

**2.3** Transform, style, and enhance text and graphics

**2.3.1** Graphics effects (rounded corners, shadows, transparency, background gradients, typography, and Web Open Font Format)

**2.3.1.1** Rounded corners, shadows, transparency

# Background Gradients

Gradients refer to the transition from one color to another. This transition can be executed in a linear orientation or the shape of a circle. The CSS properties used to execute these techniques can accept RGB values, hex colors, and common color names. Background gradients can also accept percentage values that specify color stops. Color stops can be used to achieve subtle color changes that are more appropriate in modern web design.

## Purpose

Upon completing this project, you will better understand the CSS properties used to create background gradients.

## Steps for Completion

1. Open **full-sized-div.html** from the Domain 2 Student folder.
2. Preview **full-sized-div.html**. Note the linear transition of colors along the y-axis.
3. On line 24, comment out the linear-gradient property for the background-image class on line 24.
4. On line 25, add a line of code to create a circle-shaped color transition. This property should accept the same color parameters as the linear gradient on line 24.
5. Take a screenshot of lines 23-40 of **full-sized-div.html**.
6. Save the screenshot as **231b-Completed**.

### Project Details

#### Project file

full-sized-div.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 2

**Topic:** Graphics Effects

**Subtopic:** Background Gradients

#### Objectives covered

##### 2 Graphics and Animation

**2.3** Transform, style, and enhance text and graphics

**2.3.1** Graphics effects (rounded corners, shadows, transparency, background gradients, typography, and Web Open Font Format)

**2.3.1.2** Background gradients

# Typography and Web Open Font Format

Common CSS properties used to style text include the following: color, font-family, font-style, font-weight, letter-spacing, and text-shadow. Using these properties, web developers can create easy-to-read and stylish headers for a webpage. In addition to these properties, a developer can use custom fonts using the @font-face selector. This selector allows a developer to import a font from a Web Open Font Format (.woff or WOFF) file and add that font to the font-family property.

## Purpose

Upon completing this project, you will better understand how to import fonts using the @font-face selector and a WOFF file.

## Steps for Completion

1. Open **about.html** and **main.css** from the Domain 2 Student folder.
2. On line 51 of **main.css**, add a source property linked to the **titillium-regular-webfont.woff2** file in the fonts folder. (Hint: When specifying the URL, remember to move up a folder path before moving into the fonts folder)
3. Add the newly created font to the font-family property on line 56. This font should be the primary font.
4. Preview **about.html** to verify that the font has been changed.
5. Take a screenshot of lines 49-69 of **main.css**.
6. Save the screenshot as **231c-Completed**.

### Project Details

#### Project file

about.html  
main.css

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 2

**Topic:** Graphics Effects

**Subtopic:** Typography and WOFF

#### Objectives covered

##### 2 Graphics and Animation

**2.3** Transform, style, and enhance text and graphics

**2.3.1** Graphics effects (rounded corners, shadows, transparency, background gradients, typography, and Web Open Font Format)

**2.3.1.3** Typography and Web Open Font Format

# 2D and 3D Transformations

CSS provides the ability to add both 2D and 3D transformations to elements. A developer can move, resize, skew, and rotate elements. A developer can also apply 3D changes to an element. This project will cover the process of rotating an image along the x or y-axis. Each of these transformations is achieved by using the transform property along with a CSS function.

## Purpose

Upon completing this project, you will better understand how to apply 2D and 3D transformations to HTML elements.

## Steps for Completion

1. Open **transformations.html** from the Domain 2 Student folder.
2. On line 18, add a CSS property that moves the image to the right by 200px.
3. On line 21, add a CSS property that doubles the size of the element.
4. On line 28, add a CSS property that rotates the element counterclockwise by 45 degrees.
5. On line 34, add a CSS property for a 3D rotation on the y-axis. The parameter should be 180 degrees.
6. Preview the transformations on **transformations.html**.
7. Take a screenshot of lines 17-35 of **transformations.html**.
8. Save the screenshot as **232-Completed**.

## Project Details

### Project file

transformations.html

### Estimated completion time

10 minutes

### Video reference

#### Domain 2

**Topic:** Graphics Effects

**Subtopic:** 2D and 3D Transformations

### Objectives covered

**2** Graphics and Animation

**2.3** Transform, style, and enhance text and graphics

**2.3.2** 2-D and 3-D transformations (translate, scale, rotate, skew, and 3-D perspective transitions and animations)



# Keyframes

Keyframes define CSS animations by defining several stopping points throughout an animation. Any number of CSS properties can be defined in any number of keyframes. Each animation should include a starting point (0%) and an ending point (100%).

## Purpose

Upon completing this project, you will better understand how to implement a CSS animation using the @keyframes selector.

## Steps for Completion

1. Open **keyframes.html** from the Domain 2 Student folder.
2. On line 25, add a line of code to link the image to the identifier keyframe.
3. On line 26, add a line of code to set the animation duration to 3 seconds.
4. On line 28, set the keyframe animation to loop infinitely.
5. Take a screenshot of lines 9-29 of **keyframes.html**.
6. Save the screenshot as **233-Completed**.

### Project Details

#### Project file

keyframes.html

#### Estimated completion time

15 minutes

#### Video reference

##### Domain 2

**Topic:** Graphics Effects

**Subtopic:** Keyframes

#### Objectives covered

**2** Graphics and Animation

**2.3** Transform, style, and enhance text and graphics

**2.3.3** Keyframes

# Domain 2

# Lesson 5

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 2 Lesson 5, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Grayscale is useful for lowering \_\_\_\_\_ on an image. This technique is frequently used for large backgrounds. [Grayscale]
2. The pixel value passed into the blur CSS filter function defines how many pixels are \_\_\_\_\_ to cause the blur effect. [Blur]
3. Sepia refers to a brownish-red filter, commonly associated with \_\_\_\_\_ images. [Sepia]
4. Opacity is a CSS filter function that makes an image \_\_\_\_\_. [Opacity]
5. A drop-shadow should be applied to a \_\_\_\_\_ file, which allows developers to use a transparent background. [Drop-Shadow]
6. Values over \_\_\_\_\_ are necessary to over-saturate an image. [Saturate]

# CSS Image Filters

This section will focus on the filters that can be applied using CSS. While these effects can be achieved with photo-editing software or even modern mobile devices, CSS allows developers to change the filters programmatically based on some data from a user or the browser.

## Purpose

Upon completing this project, you will better understand how to apply different CSS image filters.

## Steps for Completion

1. Open **image-filters.html** from the Domain 2 Student folder.
2. On line 16, add a line of code that removes half of the color from the image.
3. On line 19, add a line of code that blurs the image. The value should be 5 pixels.
4. On line 22, add a line of code that converts the image to an image with 80% sepia colors.
5. On line 25, add a line of code that makes the image 50% transparent.
6. Preview **image-filters.html** to verify that the filters are applied.
7. Take a screenshot of lines 9-26 of **image-filters.html**.
8. Save the screenshot as **241-Completed**.

### Project Details

#### Project file

image-filters.html

#### Estimated completion time

15 minutes

#### Video reference

##### Domain 3

**Topic:** CSS Filters on Images

**Subtopic:** Grayscale; Blur; Sepia; Opacity

#### Objectives covered

**2** Graphics and Animation

**2.4** Apply CSS filters to images

**2.4.1** Grayscale

**2.4.2** Blur

**2.4.3** Sepia

**2.4.4** Opacity

# CSS Drop Shadow

CSS drop shadow applies a shadow to the outline of an image. The drop-shadow property should not be confused with the box-shadow property. The box-shadow creates a rectangle shadow around an image regardless of whether an image has a transparent background. For an example of an image with a transparent background, preview the tennis-cut1.png image in the images folder.

## Purpose

Upon completing this project, you will better understand how to apply the drop-shadow filter to a PNG image with a transparent background.

## Steps for Completion

1. Open **image-filters.html** from the Domain 2 Student folder.
2. On line 31, add a drop shadow to the image. The shadow should be positioned five pixels down and five pixels to the right of the image. The image should have a blur radius of five pixels. The color of the shadow should be yellowgreen.
3. On line 34, add a line of code to over-saturate the image. The value passed should be ten.
4. Preview **image-filters.html** to verify that the filters are applied.
5. Take a screenshot of lines 27-35 of **image-filters.html**.
6. Save the screenshot as **245-Completed**.

### Project Details

#### Project file

image-filters.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 3

**Topic:** CSS Filters on Images

**Subtopic:** Drop-Shadow; Saturate

#### Objectives covered

**2** Graphics and Animation

**2.4** Apply CSS filters to images

**2.4.5** Drop-shadow

**2.4.6** Saturate



# Domain 3

# Lesson 1

**HTML5**  
Application Development



# Fill-in-the-Blanks

**Instructions:** While watching Domain 3 Lesson 1, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. The \_\_\_\_\_ tag can be thought of as a form title. [Form Elements]
2. The meter tag displays a meter based on its \_\_\_\_\_ attribute, not the inner HTML. [Form Elements]
3. The \_\_\_\_\_ tag is used to display the result of a calculation or script. [Form Elements]
4. The pattern attribute uses \_\_\_\_\_ to validate user input. [Pattern Attributes]
5. The length of an input can be limited by using the \_\_\_\_\_ attribute. [Length]
6. Requiring certain fields can protect against \_\_\_\_\_ form submissions. [Required Value]

# Form Elements

When building web applications, it is important to understand the process of building forms. Forms are the primary way by which users request and submit information. A clear and concise form requires significant work on the part of a developer. This section will focus on a few form elements that help build organized and easy-to-use web forms.

## Purpose

Upon completing this project, you will better understand the different elements used to build forms for web applications.

## Steps for Completion

1. Open **contact.html** from the Domain 3 Student folder.
2. On line 53, add the id attribute to the datalist tag to link it to the input tag on line 52.
3. On lines 54-57, add four options to the datalist with the following values: Fairfax, Foxboro, Valley, and Johnston.
4. Preview **contact.html**. Select the county input field to verify the datalist options.
5. Take a screenshot of lines 51-58 of **contact.html**.
6. Save the screenshot as **311-Completed**.

### Project Details

#### Project file

contact.html

#### Estimated completion time

15 minutes

#### Video reference

##### Domain 3

**Topic:** Form Markup

**Subtopic:** Form Elements

#### Objectives covered

##### 3 Forms

**3.1** Construct and analyze markup that uses form elements

**3.1.1** Datalist, fieldset, meter, legend, output

# Input Validation

A developer is expected to be able to validate the data that users input into a form. These techniques help protect against SQL injection attacks, cross-site scripting, and other injection attacks. Using these attributes, a developer can have near-total control over what data is accepted by a form. In addition to security threats, failure to validate input can lead to disorganization and inefficiency in a database. For the exam, understand that it is the job of a developer to work alongside security personnel and utilize these attributes to keep company assets safe and organized.

## Purpose

Upon completing this project, you will better understand how to implement input validation using HTML attributes.

## Steps for Completion

1. Open **contact.html** from the Domain 3 Student folder.
2. On line 46, edit the input tag to be the correct input data type and make it a required field.
3. On line 61, specify the pattern attribute that will validate the input field only if there are five digits.
4. On line 65, add an attribute to the textarea tag that limits the input to 255 characters.
5. Take a screenshot of lines 46-67 of **contact.html**.
6. Save the screenshot as **321-Completed**.

### Project Details

#### Project file

contact.html

#### Estimated completion time

15 minutes

#### Video reference

##### Domain 3

**Topic:** Configure Input Validation

**Subtopic:** Validation Attributes;  
Pattern Attributes; Correct Data  
Types; Length; Required Value

#### Objectives covered

##### 3 Forms

**3.2** Configure input validation

**3.2.1** Validation attributes

**3.2.2** Pattern attribute for regular  
expressions

**3.2.3** Correct data type

**3.2.4** Length

**3.2.5** Required value

# Domain 4

# Lesson 1

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 4 Lesson 1, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. \_\_\_\_\_ elements occupy the full width of a container. [Content Flow]
2. Floating an element left or right removes that element from the normal \_\_\_\_\_ of a webpage. [Element Positioning]
3. \_\_\_\_\_ is placeholder text that is frequently used to fill text regions during a design process. [Content Overflow]
4. A CSS \_\_\_\_\_ is a list of CSS rule sets designed to address inconsistencies among browsers. [Basic CSS Styling]

# Content Flow

Content flow refers to the relationship between elements as they are rendered to a webpage. Elements can display side-by-side, centered, or removed from the flow of the page altogether. This project will focus on the display property as it relates to content flow.

## Purpose

Upon completing this project, you will better understand how the display property affects elements on a webpage.

## Steps for Completion

1. Open **flow.html** from the Domain 4 Student folder.
2. On line 22, add a line of code that will make all the items ignore the width and height values and resize the elements to fit the content. These items should display side-by-side filling the container.
3. Preview **flow.html** to view the changes.
4. Comment out the code on line 22.
5. On line 23, add a line of code that will specify that the items display side-by-side across the container but keep their width and height values.
6. Take a screenshot of lines 18-24 of **flow.html**.
7. Save the screenshot as **411-Completed**.

### Project Details

#### Project file

flow.html

#### Estimated completion time

10 minutes

#### Video reference

#### Domain 4

**Topic:** Layout, Positioning, and Flow

**Subtopic:** Content Flow

#### Objectives covered

#### 4 Layouts

**4.1** Manage content layout, positioning, and flow by using CSS

**4.1.1** Content flow (inline vs. block flow)



# Positioning

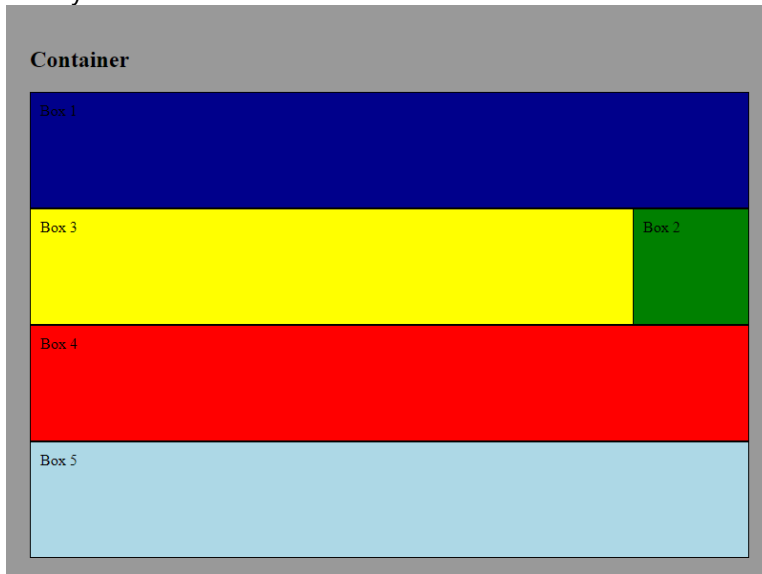
In addition to specifying the natural flow of elements using the display property, developers can also float and position elements relative to the flow of elements and the container itself. This project focuses on the CSS properties used to float and position elements within a webpage's flow.

## Purpose

Upon completing this project, you will better understand the float property and the position property as they relate to positioning elements on a webpage.

## Steps for Completion

1. Open **positioning.html** from the Domain 4 Student folder.
2. Preview **positioning.html** in a browser.
3. On line 37, add a line of code to the #box2 selector that would create the layout below:



4. Preview **positioning.html** in a browser to verify the layout.
5. Take a screenshot of lines 35-38 of **positioning.html**.
6. Save the screenshot as **412-Completed**.

## Project Details

### Project file

positioning.html

### Estimated completion time

5 minutes

### Video reference

#### Domain 4

**Topic:** Layout, Positioning, and Flow

**Subtopic:** Element Positioning

### Objectives covered

#### 4 Layouts

**4.1** Manage content layout, positioning, and flow by using CSS

**4.1.2** Positioning of individual elements (float vs. absolute positioning)

# Content Overflow and CSS Reset

CSS offers developers the opportunity to specify how the browser should handle content that overflows its element. Handling content that overflows its element is accomplished by specifying the overflow property for an element. This project will review the four possible values for the overflow property.

A CSS Reset is a list of CSS rule sets designed to address inconsistencies among browsers. One of the most popular rule sets is Eric Meyer's CSS reset, which is short, simple, and free to use.

## Purpose

Upon completing this project, you will better understand how a developer can use CSS to handle content overflow. You will also better understand the purpose of a CSS reset.

## Steps for Completion

1. Match the overflow property value with its description.

|           |            |
|-----------|------------|
| A. Scroll | C. Visible |
| B. Hidden | D. Auto    |

- \_\_\_\_\_ Handles overflow based on the container and the content
  - \_\_\_\_\_ Adds X and Y scroll bars to an element
  - \_\_\_\_\_ No scroll bars. Overflow content is not rendered in the browser
  - \_\_\_\_\_ Overflow content renders outside of an element. This content will overlap other elements
2. Open **main.css** from the Domain 4 Student folder.
  3. List the last four HTML elements that are listed in the opening rule set of the CSS reset.
    - \_\_\_\_\_
    - \_\_\_\_\_
    - \_\_\_\_\_
    - \_\_\_\_\_

## Project Details

### Project file

main.css

### Estimated completion time

5 minutes

### Video reference

#### Domain 4

**Topic:** Layout, Positioning, and Flow  
**Subtopic:** Content Overflow; Basic CSS Styling

### Objectives covered

#### 4 Layouts

- 4.1** Manage content layout, positioning, and flow by using CSS
  - 4.1.3** Content overflow (scrolling, visible, and hidden)
  - 4.1.4** Basic CSS styling

# Domain 4

# Lesson 2

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 4 Lesson 2, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Grid-view is a technique that involves a \_\_\_\_\_ column layout, like the one seen in CSS frameworks like Bootstrap. [Grid View]
2. A background-size property set to \_\_\_\_\_ will fill the entire element, even if the image is slightly cut off or made larger. [Background Size]
3. In most cases, when sizing an image, \_\_\_\_\_ values should be used to create responsive images. [Images and Pictures]
4. The viewport is the \_\_\_\_\_ area on a device. [Viewports]

# Grid View

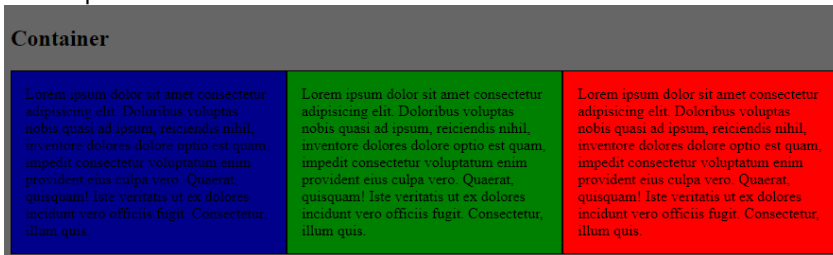
Grid view is a technique that involves using a 12-column layout, like the one used in the popular CSS framework, Bootstrap. Using this method, developers can easily adjust the content layout based on the width of the screen. When used in conjunction with media queries, this technique can design responsive page layouts with little code.

## Purpose

Upon completing this project, you will better understand CSS Grid View and how it can support a responsive web design.

## Steps for Completion

1. Open **grid-view.html** from the Domain 4 Student folder.
2. Preview **grid-view.html** in a browser. The three elements are stacked vertically.
3. On lines 72, 78, and 85, add a class to each div that specifies each div to display side-by-side in the row, dividing the container width into three equal parts.
4. Preview **grid-view.html**. The elements should display in the same row with equal width:



5. Take a screenshot of lines 70-92 of **grid-view.html**.
6. Save the screenshot as **421-Completed**.

## Project Details

### Project file

grid-view.html

### Estimated completion time

10 minutes

### Video reference

#### Domain 4

**Topic:** Responsive Design

**Subtopic:** Grid View

### Objectives covered

#### 4 Layouts

**4.2** Construct layouts by using responsive design

**4.2.1** Grid view

# Background Size

The background-size property specifies how a web browser should display an image as a webpage’s background. This property can also specify backgrounds for specific elements. Backgrounds are vital to design because, without a proper background, a webpage can feel bland. However, if backgrounds are too busy, they can distract from the main content. Developers must take control of all elements of a background.

## Purpose

Upon completing this project, you will better understand the background-size property and its property values.

## Steps for Completion

1. Briefly describe the following property values for the CSS background-size property.
  - a. Auto: \_\_\_\_\_
  - b. Cover: \_\_\_\_\_
  - c. Contain: \_\_\_\_\_
2. Indicate whether the following statement is true or false. False.
  - a. \_\_\_\_\_ Content that is not amiable to change is likely to look good across multiple devices.

### Project Details

#### Project file

N/A

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 4

**Topic:** Responsive Design

**Subtopic:** Background Size

#### Objectives covered

##### 4 Layouts

**4.2** Construct layouts by using responsive design

**4.2.2** Background-size



# Images

Images are a very large part of modern web design. In many cases, designers and photographers go to great lengths to deliver high-quality images with which developers can work. For that reason, developers must know how to present those images in the best possible way. Optimizing how images display on a webpage often starts with using relative measurements, such as a percentage, to size an image.

## Purpose

Upon completing this project, you will better understand how relative measurements relate to responsive web design.

## Steps for Completion

1. Open **image.html** from the Domain 4 Student folder.
2. Preview **image.html** in a browser. The image will appear very large.
3. On lines 10-13 of **image.html**, add three lines of code to specify the image display as a block-level element, a top and bottom margin of 0, left and right margins of auto, and a width of 90% of the container.
4. Take a screenshot of lines 10-13 of **image.html**.
5. Save the screenshot as **423-Completed**.

### Project Details

#### Project file

image.html

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 4

**Topic:** Responsive Design

**Subtopic:** Images and Pictures

#### Objectives covered

##### 4 Layouts

**4.2** Construct layouts by using responsive design

**4.2.3** Images

**4.2.4** Picture

# Viewport

The head section contains metadata for the webpage. In each HTML file in this course, you will find a meta tag with a name attribute set to viewport. Then a content attribute set to width=device-width, initial-scale=1.0. This tag is included on each page because it helps mobile browsers and smaller screens render pages properly.

## Purpose

Upon completing this project, you will better understand the viewport meta tag and the role it plays in responsive web design.

## Steps for Completion

1. Open **about.html** from the Domain 4 Student folder.
2. Review the head tag of **about.html**. Make the appropriate corrections to the viewport metatag.
3. Save the screenshot as **425-Completed**.

### Project Details

#### Project file

about.html

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 4

**Topic:** Responsive Design

**Subtopic:** Viewports

#### Objectives covered

##### 4 Layouts

**4.2** Construct layouts by using responsive design

**4.2.5** Viewport

# Domain 4

# Lesson 3

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 4 Lesson 3, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Viewwidth (vw) and viewheight (vh) units are based on the size of the \_\_\_\_\_, whereas percentage units are based on the size of the parent container. [Responsive Width]
2. Media queries should be added \_\_\_\_\_ any previous code to avoid styles being overwritten. [Media Queries]
3. The \_\_\_\_\_ is the most common value used to determine whether a media query will run. [Media Queries]
4. In the past, developers relied on \_\_\_\_\_ to organize page layouts. The position, float, and display attributes were heavily utilized. [Flex Container]
5. The \_\_\_\_\_ property sets the baseline size for a flex item. [Flex Items]

# Responsive Width and Media Queries

Responsive width refers to a value given to an HTML element based on some other factor. Viewwidth and viewheight specify a value relative to the size of the viewport. A percentage value specifies a value based on the parent container.

Media queries are used to specify a different collection of CSS rule sets based on some other factor. In most cases, the max-width of a screen is used to determine whether these blocks of code run. As a best practice, media queries should be placed at the end of a stylesheet to avoid styles being overwritten.

## Purpose

Upon completing this project, you will better understand responsive width and the syntax for media queries.

## Steps for Completion

1. Open **mediaquery.html** from Domain 4 Student folder.
2. Preview **mediaquery.html** in a full-screen browser. The image displayed is 800px wide.
3. On lines 17-21, add a media query that activates when the screen has a maximum width of 850px. Inside the media query, the image tag should have a width of 80% of the viewport window.
4. Preview **mediaquery.html**. Resize the browser window. When the viewport is larger than 850px wide, the image should be 800px wide. When the viewport is less than 850px wide, the image should have a responsive width.
5. Take a screenshot of lines 8-22 of **mediaquery.html**.
6. Save the screenshot as **426-Completed**.

### Project Details

#### Project file

mediaquery.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 4

**Topic:** Responsive Design

**Subtopic:** Responsive Width;  
Media Queries

#### Objectives covered

##### 4 Layouts

**4.2** Construct layouts by using responsive design

**4.2.6** Responsive width

**4.2.7** Media queries

# CSS Flexbox

In the early stages of the internet, web developers relied on tables to lay out webpages. They also relied heavily on the position, float, and display properties. In modern design, CSS flexbox has proven to be a superior means of laying out a webpage.

CSS flexbox allows a developer to create a two-dimensional layout based on either columns or rows. These layouts can be simple to create and flexible across devices using several CSS flex properties.

## Purpose

Upon completing this project, you will better understand CSS flexbox.

## Steps for Completion

1. Open **flexbox.html** from the Domain 4 Student folder.
2. Preview **flexbox.html** in a browser. The flex items should be displayed as block-level elements.
3. On lines 19 and 20, add two lines of code. The first line should specify that the parent class function as a flex container. The second line should specify that flex items be displayed in a row and flex items be wrapped to maintain their square dimensions.
4. Take a screenshot of lines 16-22 of **flexbox.html**.
5. Save the screenshot as **431-Completed**.

### Project Details

#### Project file

flexbox.html

#### Estimated completion time

10 minutes

#### Video reference

#### Domain 4

**Topic:** Flexible Responsive Layouts

**Subtopic:** Flex Container

#### Objectives covered

#### 4 Layouts

**4.3** Construct flexible, responsive layouts by using CSS flexbox

**4.3.1** Flex container (flex-direction, flex-flow, flex-wrap)



# CSS Flex Items

This project will focus on the different CSS properties developers can use to customize the items in a flex container. Flex items are the direct children of a container that has a display property of flex. Flex items are typically HTML div elements. This project will focus on using CSS properties to customize the relationship between items as a container grows and shrinks.

## Purpose

Upon completing this project, you will better understand how to specify common properties for flex items.

## Steps for Completion

1. Open **flexbox-items.html** from the Domain 4 Student folder.
2. On lines 60-64, add an inline style to each flex item with the following properties:
  - a. Flex grow for each item should be set to 0
  - b. Flex-shrink should be set to 1 for all items except the third item. That item should have a flex-shrink value of 5
  - c. Flex basis for each item should be set to 100px
3. Take a screenshot of lines 59-65 of **flexbox-items.html**.
4. Save the screenshot as **432-Completed**.

### Project Details

#### Project file

flexbox-items.html

#### Estimated completion time

15 minutes

#### Video reference

##### Domain 4

**Topic:** Flexible Responsive Layouts

**Subtopic:** Flex Items

#### Objectives covered

##### 4 Layouts

**4.3** Construct flexible, responsive layouts by using CSS flexbox

**4.3.2** Flex items (flex-basis, flex-grow, flex-shrink, order, flex)

# Domain 4

# Lesson 4

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 4 Lesson 4, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. A \_\_\_\_\_ unit is a relative unit that can be used in place of percentage values, ensuring that content fills a grid container. [Containers]
2. The \_\_\_\_\_ property is used to span items across multiple columns. [Items]
3. The \_\_\_\_\_ property is used to specify a unique name for an element. That name can then be used in the grid-template-areas property to define a layout. [Templates]
4. Grid lines refer to specific lines between grid items, whereas grid gap refers to the amount of \_\_\_\_\_ between grid areas. [Gaps and Lines]

# Grid Container

CSS Grids allow a developer to lay out content based on both columns and rows. Like CSS flexbox, grids allow a developer to create flexible page layouts without relying upon tables, floats, and positioning.

## Purpose

Upon completing this project, you will better understand how to design a simple layout using CSS Grids.

## Steps for Completion

1. Open **grids1.html** from the Domain 4 Student folder.
2. On line 16, add a line of code to organize the grid into three columns. The first two columns should occupy 25% of the row. The last column should occupy 50% of the row.
3. Preview **grids1.html** in a browser. Ensure that the columns are divided like the example below:



4. Take a screenshot of lines 11-17 of **grids1.html**.
5. Save the screenshot as **441-Completed**.

## Project Details

### Project file

grids1.html

### Estimated completion time

10 minutes

### Video reference

#### Domain 4

**Topic:** Grid-Based Layouts

**Subtopic:** Containers

### Objectives covered

#### 4 Layouts

**4.4** Construct grid-based layouts by using CSS grid

**4.4.1** Container

# Grid Items

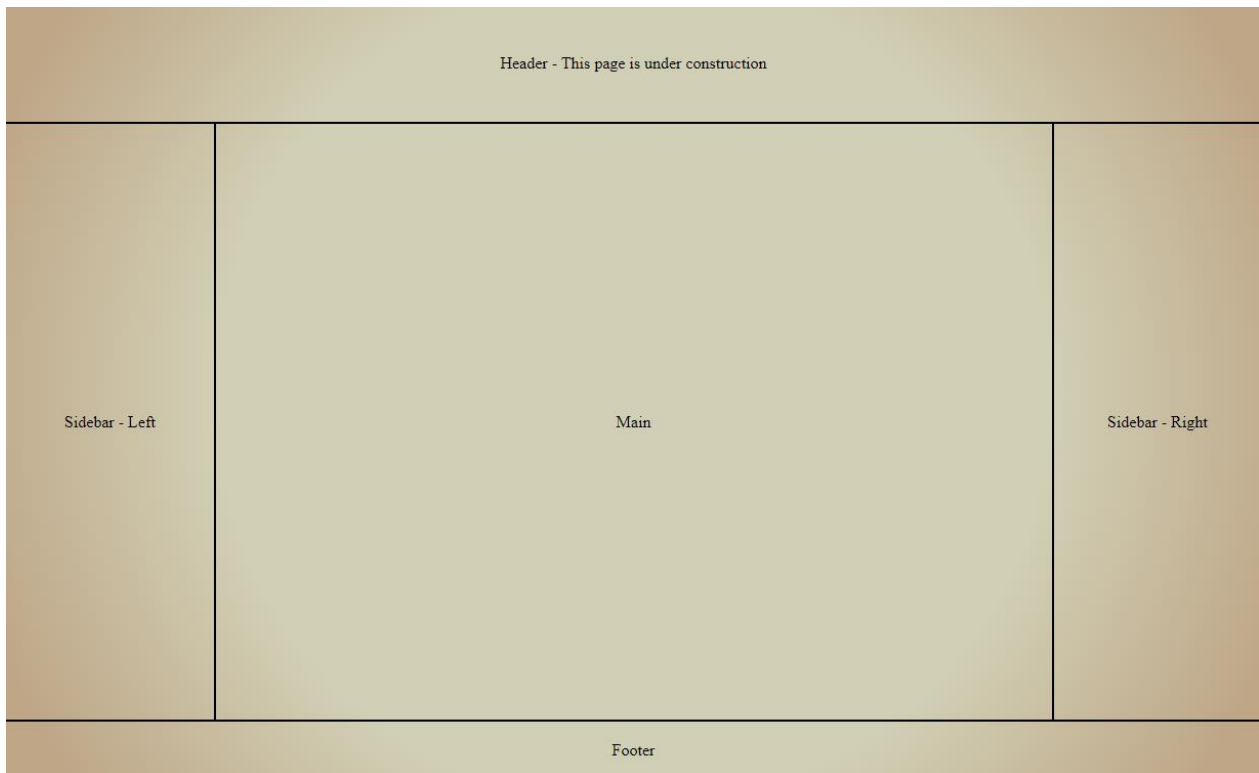
HTML div elements inside of a grid container are known as grid items. Grid items can be stretched across grid columns or grid rows. Stretching elements across a grid is one technique a developer can use to fill a grid with content. This project will focus on the grid-column property, but the grid-row property functions similarly.

## Purpose

Upon completing this project, you will better understand how to fit content inside of a grid layout.

## Steps for Completion

1. Open **blog.html** from the Domain 4 Student folder.
2. Preview **blog.html** in a browser. The grid items are not properly placed within the grid.
3. On lines 37 and 43, add code to stretch the header and footer across all columns, fixing the page layout.
4. Preview **blog.html** in a browser. Ensure that the sections are divided like the example below:



5. Take a screenshot of lines 38-46 of **blog.html**.
6. Save the screenshot as **442-Completed**.

## Project Details

### Project file

blog.html

### Estimated completion time

10 minutes

### Video reference

#### Domain 4

**Topic:** Grid-Based Layouts

**Subtopic:** Items

### Objectives covered

#### 4 Layouts

**4.4** Construct grid-based layouts by using CSS grid

**4.4.2** Items

# Grid Templates

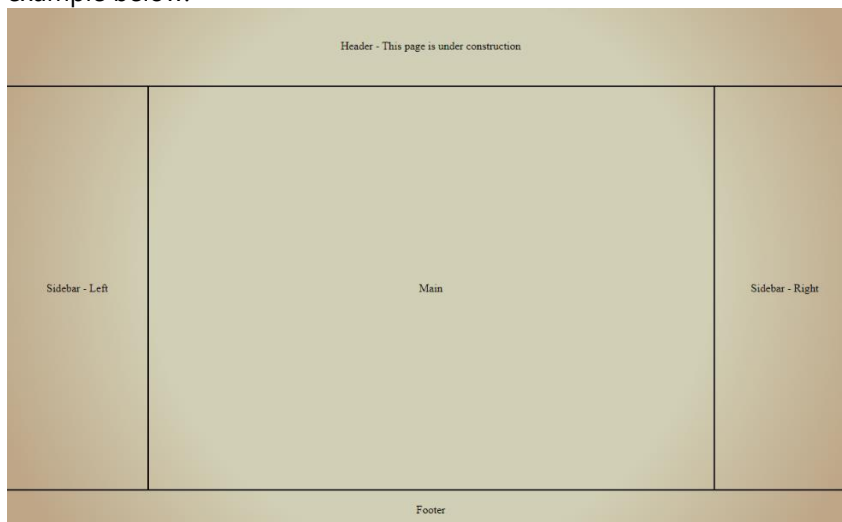
Grid layouts can also be organized using grid templates. The grid-area property is first applied to individual grid items. This property specifies a name for a grid item. This name can then be used in the grid-template property of a grid container to organize content. In the grid template property, rows are divided by single or double quotes, and there are no commas. It is recommended that these rows be oriented in the code as they are on the page, making them easier to read.

## Purpose

Upon completing this project, you will better understand how to organize content in a grid layout using a grid template.

## Steps for Completion

1. Open **blog1.html** from the Domain 4 Student folder.
2. On line 26, add code to layout the grid items so that it displays like the example below:



3. Take a screenshot of lines 19-29 of **blog1.html**.
4. Save the screenshot as **443-Completed**.

## Project Details

### Project file

blog1.html

### Estimated completion time

10 minutes

### Video reference

#### Domain 4

**Topic:** Grid-Based Layouts

**Subtopic:** Templates

### Objectives covered

#### 4 Layouts

**4.4** Construct grid-based layouts by using CSS grid

**4.4.3** Templates



# Grid Gaps and Lines

Grids can be customized even further with a better understanding of gaps and lines. Grid gaps simply specify the space between grid items. Space can be specified for both columns and rows. Grid lines refer to a line that separates grid areas. Do not forget that grid lines are not zero-based, meaning that the first grid line in a layout will be labeled with the number 1.

## Purpose

Upon completing this project, you will better understand grid gaps and grid lines.

## Steps for Completion

1. Open **blog.html** from the Domain 4 Student folder.
2. On line 26, add a single CSS property to specify a 10px column gap and row gap for the grid container.
3. Take a screenshot of lines 19-27. Save the screenshot as **444a-Completed**.
4. Preview **blog.html** in a browser. Press F12 to open the Developer Tools.
5. Under the Elements tab, select the Grid icon.
6. Take a screenshot of the line numbers displayed on the Page Preview.
7. Save the screenshot as **444b-Completed**.

### Project Details

#### Project file

blog.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 4

**Topic:** Grid-Based Layouts

**Subtopic:** Gaps and Lines

#### Objectives covered

##### 4 Layouts

**4.4** Construct grid-based layouts by using CSS grid

**4.4.4** Gap/Lines

# Domain 5

# Lesson 1

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 5 Lesson 1, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Instantiation is the creation of a JavaScript object based on a template or \_\_\_\_\_. [Instantiation]
2. Looping through objects and arrays and retrieving values from different objects is a common part of \_\_\_\_\_ programming. [Properties]
3. The \_\_\_\_\_ keyword allows developers to reuse methods based on the object that is calling the method. [Methods]
4. The \_\_\_\_\_ keyword is used in JavaScript to inherit one class from another. [Inheritance]

# Instantiation

Instantiation is the creation of an object based on a JavaScript template or class. The characteristics that define an object are outlined in a constructor function. Defining a class using a constructor function does not itself instantiate an object, like how a function can be defined but not run until properly called.

## Purpose

Upon completing this project, you will better understand how to instantiate objects in JavaScript.

## Steps for Completion

1. Open **index.html** and **classes.js** from Domain 5 Student folder.
2. On line 17 of **classes.js**, instantiate a new player object. Store the new object in a variable called player4. This new player should be named Dianna Thompson. She is ten years old.
3. Take a screenshot of lines 14-20 of **classes.js**.
4. Save the screenshot as **511a-Completed**.
5. Preview **index.html**. Find the new object in the baseballTeam array that is logged to the console.
6. Take a screenshot documenting the new object.
7. Save the screenshot as **511b-Completed**.

### Project Details

#### Project file

index.html  
classes.js

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 5

**Topic:** Custom Classes

**Subtopic:** Instantiation

#### Objectives covered

**5** JavaScript Coding

**5.1** Create and use custom classes

**5.1.1** Instantiation

# Properties

As a developer, it is important to understand how to retrieve properties from objects. In most cases, this will involve retrieving several properties at one time. This project focuses on using a simple for-loop to retrieve like properties from an array of objects.

## Purpose

Upon completing this project, you will better understand how to retrieve properties from JavaScript objects.

## Steps for Completion

1. Open **index.html** and **classes.js** from the Domain 5 Student folder.
2. On line 25, add a for-loop that logs each player's first name in the `baseballTeam` object array to the console.
3. Take a screenshot of the for-loop on line 25.
4. Save the screenshot as **512-Completed**.

### Project Details

#### Project file

index.html  
classes.js

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 5

**Topic:** Custom Classes

**Subtopic:** Properties

#### Objectives covered

**5** JavaScript Coding

**5.1** Create and use custom classes

**5.1.2** Properties

# Methods

In addition to properties, JavaScript objects can store methods. Using the `this` keyword, methods can be designed to use property values from an object itself. The `this` keyword allows developers to write one method which can return different data based on the object that calls the method. This project will demonstrate how the same method can return different strings based on the object that calls the method.

## Purpose

Upon completing this project, you will better understand how to call object methods.

## Steps for Completion

1. Open **index.html** and **classes.js** from the Domain 5 Student folder.
2. On line 31, add a line of code that calls the greeting method using the `player1` object. Log the results to the console.
3. On line 32, add a line of code that calls the greeting function using the `player3` object. Log the results to the console.
4. Take a screenshot of lines 31 and 32.
5. Save the screenshot as **513a-Completed**.
6. Preview **index.html** and open the console.
7. Take a screenshot of the console that includes the two messages.
8. Save the screenshot as **513b-Completed**.

### Project Details

#### Project file

index.html  
classes.js

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 5

**Topic:** Custom Classes

**Subtopic:** Methods

#### Objectives covered

**5** JavaScript Coding

**5.1** Create and use custom classes

**5.1.3** Methods



# Inheritance

In object-oriented programming, the concept of inheritance describes the process of creating a class that inherits all properties and methods from another class. Using class inheritance is a technique that developers can use to avoid rewriting code.

## Purpose

Upon completing this project, you will better understand the concept of inheritance as it applies to object-oriented programming.

## Steps for Completion

1. Open **index.html** and **classes.js** from the Domain 5 Student folder.
2. On line 45, instantiate a new Coach object and store that object in a variable called coach1. This new coach is named Nicky Moon, and he is 34 years old.
3. On line 46, add the coach1 object into the baseballTeam array.
4. On line 47, using the coach1 object and its removePlayer method, remove the player with the first name Bob from the baseballTeam array.
5. On line 48, log the baseballTeam array to the console to verify the removal of the player.
6. Preview **index.html** and open the Developer Tools. Confirm the addition of coach1 and the removal of the player with the first name Bob from the array.
7. Take a screenshot of lines 45-48 of **classes.js**.
8. Save the screenshot as **514-Completed**.

### Project Details

#### Project file

index.html  
classes.js

#### Estimated completion time

15 minutes

#### Video reference

##### Domain 5

**Topic:** Custom Classes

**Subtopic:** Inheritance

#### Objectives covered

**5** JavaScript Coding

**5.1** Create and use custom classes

**5.1.4** Inheritance

# Domain 5

# Lesson 2

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 5 Lesson 2, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Once the readyState equals \_\_\_\_\_ and the status equals 200, a request is complete. [Send and Receive Data]
2. HTML displays content in a browser, whereas XML is meant to store and \_\_\_\_\_ data. [XML]
3. XML is organized in a \_\_\_\_\_ structure, like the DOM. [XML]

# Sending and Receiving Data

JavaScript features several methods of sending and receiving data. One such method uses the XMLHttpRequest object. Despite the naming, this object can be used to transmit XML and JSON data. The XMLHttpRequest object is used to achieve a concept called AJAX, Asynchronous JavaScript, and XML. In short, AJAX enables a webpage to load content while a server request is made in the background. Without AJAX, JavaScript would be unable to move through to other code until a request was completed.

## Purpose

Upon completing this project, you will better understand how to send and receive data using the XMLHttpRequest object in JavaScript.

## Steps for Completion

1. Open **ajax.html** from the Domain 5 Student folder.
2. On line 48, initialize a new request. The request method should be set to GET, and the URL should retrieve the example.txt file from the text folder.
3. On line 51, send the XMLHttpRequest.
4. Take a screenshot of lines 47-52 of **ajax.js**.
5. Save the screenshot as **521-Completed**.

### Project Details

#### Project file

ajax.html

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 5

**Topic:** Data Access Through JavaScript

**Subtopic:** Send and Receive Data

#### Objectives covered

##### 5 JavaScript Coding

**5.2** Perform data access by using JavaScript

**5.2.1** Send and receive data

# XML

XML stands for Extensible Markup Language. XML is formatted similarly to HTML; however, XML serves a much different purpose. XML is designed to store and transfer data. XML tags can be customized based on the data, whereas HTML tags are predefined. XML is organized as a tree structure, like the Document Object Model. As developers, it is vital to know the anatomy of an XML file and how to retrieve appropriate data from such a file.

## Purpose

Upon completing this project, you will better understand how to use XML data.

## Steps for Completion

1. Open **xmlData1.html** from the Domain 5 Student folder.
2. On line 34, add a line of code that stores the responseXML from the request object in the variable called xmlObj.
3. On line 35, add a line of code to log the xmlObj variable to the console.
4. Take a screenshot of lines 27-42 of **xmlData1.html**.
5. Save the screenshot as **522a-Completed**.
6. Preview **xmlData1.html** in a browser and open the console. Select the #document drop-down arrow to view all 12 books in the catalog.
7. Take a screenshot of the XML object in the console.
8. Save the screenshot as **522b-Completed**.

## Project Details

### Project file

xmlData1.html

### Estimated completion time

10 minutes

### Video reference

#### Domain 5

**Topic:** Data Access Through JavaScript

**Subtopic:** XML

### Objectives covered

**5** JavaScript Coding

**5.2** Perform data access by using JavaScript

**5.2.2** XML



# Domain 5

# Lesson 3

**HTML5**  
Application Development



# Fill-in-the-Blanks

**Instructions:** While watching Domain 5 Lesson 3, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. JavaScript Object Notation (JSON) data is made up of \_\_\_\_\_ pairs. [JSON]
2. JSON can accept strings, integers, Boolean values, and even \_\_\_\_\_. [JSON]
3. The \_\_\_\_\_ method is used to parse a JSON string into a JavaScript object. [JSON]
4. To access objects in large files, developers can use a \_\_\_\_\_ to execute code on each object in the data. [Transmit and Parse Complex Objects]
5. The \_\_\_\_\_ object can be used to read files from a local system. [Load and Save Files]

# JSON

JSON is a very popular data format due to its readability and usability with JavaScript and other programming languages. JSON data is stored as text in what is called JSON strings. These strings must be parsed into JavaScript objects before values can be retrieved. JSON is made up of key/value pairs.

## Purpose

Upon completing this project, you will better understand how to work with JSON data using JavaScript.

## Steps for Completion

1. Open **jsonData1.html** from the Domain 5 Student folder.
2. On line 28, initialize a variable called `player`. This variable should be a string literal that represents JSON data. Using key/value pairs, store the following information:
  - a. The player's name is Debby.
  - b. She plays softball.
  - c. She is 14 years old.
  - d. She is not a new player.
3. Below the `player` variable, parse the `player` string into a JSON object and store it in a variable called `parsed`.
4. Take a screenshot of the `player` and `parsed` variables.
5. Save the screenshot as **523-Completed**.

## Project Details

### Project file

jsonData1.html

### Estimated completion time

10 minutes

### Video reference

#### Domain 5

**Topic:** Data Access Through JavaScript

**Subtopic:** JSON

### Objectives covered

**5** JavaScript Coding

**5.2** Perform data access by using JavaScript

**5.2.3** JSON

# Complex Objects

As a developer, you will seldom be working with single pieces of local data. It is much more common to work with large pieces of data which are received through a request. In this project, you will see an example of how a larger JSON data file can be used to create multiple elements on an HTML page.

## Purpose

Upon completing this project, you will better understand how to work with large data files in JavaScript and HTML.

## Steps for Completion

1. Open **jsonData2.html** from the Domain 5 Student folder.
2. On line 43, store the title attribute for each user in a variable called title.
3. On line 44, store a new HTML li element in a variable called li.
4. On line 45, set the `textContent` of the new element to be equal to title.
5. On line 46, append the new HTML element to the HTML element with an id of output.
6. Take a screenshot of lines 40-48.
7. Save the screenshot as **524-Completed**.

### Project Details

#### Project file

jsonData2.html

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 5

**Topic:** Data Access Through JavaScript

**Subtopic:** Transmit and Parse Complex Objects

#### Objectives covered

**5** JavaScript Coding

**5.2** Perform data access by using JavaScript

**5.2.4** Transmit and parse complex objects

# Load and Save Files

Sometimes developers need to load or save files from a local computer. In this example, we will use the FileReader object in JavaScript to retrieve a file. The File object can be used to store text files on a local computer. To see an example, review the createTextFile.html page from the Domain 5 Student folder.

## Purpose

Upon completing this project, you will better understand how to load and save files in a browser using JavaScript.

## Steps for Completion

1. Open **fileReader.html** from the Domain 5 Student folder.
2. On line 31, create a new instance of a FileReader object and store it in a reader variable.
3. Preview **fileReader.html** in a browser. Select the choose files button and select a text file. Open the console and verify the contents of the text file.
4. Take a screenshot of lines 27-38 of **fileReader.html**.
5. Save the screenshot as **525-Completed**.

### Project Details

#### Project file

fileReader.html

#### Estimated completion time

5 minutes

#### Video reference

##### Domain 5

**Topic:** Data Access Through JavaScript

**Subtopic:** Load and Save Files

#### Objectives covered

**5** JavaScript Coding

**5.2** Perform data access by using JavaScript

**5.2.5** Load and save files

# Domain 5

# Lesson 4

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 5 Lesson 4, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Event listeners can be added to HTML elements by invoking the \_\_\_\_\_ method. [Gesture Events]
2. \_\_\_\_\_, loops, and lists can help handle multiple events. [Multiple Events]
3. An \_\_\_\_\_ function has no name. [Event Object]
4. The \_\_\_\_\_ attribute for an event listener specifies how to handle bubbling and cascading events. [Bubbling vs. Cascading]



# Gesture Events

Events refer to any action by a user or any change to an HTML element. Developers can specify that certain code run because of these actions. Developers add what is called an event listener to accomplish this. This event listener reacts to an event and fires the code. This project will focus on adding a single event listener to a single HTML element.

## Purpose

Upon completing this project, you will better understand how to add an event listener to an HTML element.

## Steps for Completion

1. Open **schedule.html** and **events1.js** from the Domain 5 Student folder.
2. On line 11, add an event listener for the card2 element. The event listener should listen for a mouse click. The function should add the card-highlight class to the card2 element.
3. Preview **schedule.html** and verify the event listener functionality for the element.
4. Take a screenshot of the event listener for the card2 element from **events1.js**.
5. Save the screenshot as **531-Completed**.

## Project Details

### Project file

schedule.html  
events1.js

### Estimated completion time

5 minutes

### Video reference

#### Domain 5

**Topic:** Event Listeners and Handlers

**Subtopic:** Gesture Events

### Objectives covered

#### 5 JavaScript Coding

**5.3** Construct code that responds to events by using event listeners and handlers

**5.3.1** Gesture events

# Multiple Events

Developers typically manage many events on a webpage at any given time. Multiple events help provide a truly customized experience for the end-user. The process of adding multiple events is important as well. Developers must take full advantage of reusable code whenever possible. Reusable code helps avoid ambiguous code, logic errors, and syntax errors.

## Purpose

Upon completing this project, you will better understand how to handle multiple events using reusable code.

## Steps for Completion

1. Open **schedule.html** and **events2.js** from the Domain 5 Student folder.
2. On line 16 of **schedule.html**, comment out the link to **events1.js**. On line 17, uncomment the link to **events2.js**.
3. On lines 14-17, add an event listener to each element stored in lines 3 through 6. Each event listener should listen for a click and call the `cardClick` function.
4. Take a screenshot of lines 14-17.
5. Save the screenshot as **532-Completed**.

## Project Details

### Project file

schedule.html  
events2.js

### Estimated completion time

5 minutes

### Video reference

#### Domain 5

**Topic:** Event Listeners and Handlers

**Subtopic:** Multiple Events

### Objectives covered

#### 5 JavaScript Coding

**5.3** Construct code that responds to events by using event listeners and handlers

**5.3.2** Handling multiple events

# Event Object

Each time an event occurs, developers can retrieve an event object. This object has many properties that detail the event. Using this object and its properties, developers can create a seamless flow of information between a user and a webpage.

## Purpose

Upon completing this project, you will better understand how to access the event object.

## Steps for Completion

1. Open **index.html** and **coords.js** from the Domain 5 Student folder.
2. On line 4, add an event listener to the HTML element stored in element.
3. The event listener should listen for a click event. The function for the event listener should store a short message in a variable called msg, which specifies the offsetX and offsetY properties from the click event. That message should be displayed in the output tag.
4. Take a screenshot of lines 4 through 10.
5. Save the screenshot as **533-Completed**.
6. Describe the difference between bubbling events and cascading events.

### Project Details

#### Project file

index.html  
coords.js

#### Estimated completion time

10 minutes

#### Video reference

##### Domain 5

**Topic:** Event Listeners and Handlers

**Subtopic:** Event Object; Bubbling vs. Cascading

#### Objectives covered

##### 5 JavaScript Coding

**5.3** Construct code that responds to events by using event listeners and handlers

**5.3.3** Event object

**5.3.4** Bubbling vs. cascading

# Domain 5

# Lesson 6

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 5 Lesson 5, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. An application programming interface (API) is the link between two \_\_\_\_\_. [Google Charts]
2. Google Charts display as \_\_\_\_\_, which makes them compatible across browsers and devices. [Google Charts]
3. A \_\_\_\_\_ is used to begin a jQuery statement. [jQuery]
4. jQuery uses selectors to access elements, similar to the syntax used in \_\_\_\_\_. [jQuery]
5. The \_\_\_\_\_ method is used to find the location of a user. It accepts a function as its parameter, and that function can pass the location object. [Geolocation]

# Google Charts

The Google Charts Application Programming Interface (API) is very popular and provides developers with an easy way to visualize data without hard-coding charts using canvases or CSS. With a few parameters, any chart style can be drawn with hover effects, custom colors, and other customizations. Using a tool like the Google Charts API can save a developer time while also reducing the amount of code.

## Purpose

Upon completing this project, you will better understand how to use the Google Charts API.

## Steps for Completion

1. Open **about.html** and **chart.js** from the Domain 5 Student folder.
2. Add the following data points to the existing chart:

| Year | Memberships |
|------|-------------|
| 2017 | 15          |
| 2018 | 43          |
| 2019 | 132         |

3. Change the title of the y-axis to **Total Memberships**.
4. Take a screenshot of the data and options variables from **chart.js**.
5. Save the screenshot as **541a-Completed** to your Domain 5 Student folder.
6. Preview **about.html** in a browser. Verify the changes to the chart. Take a screenshot of the chart.
7. Save the screenshot as **541b-Completed**.

## Project Details

### Project file

about.html  
chart.js

### Estimated completion time

15 minutes

### Video reference

#### Domain 5

**Topic:** JavaScript APIs

**Subtopic:** Google Charts

### Objectives covered

**5** JavaScript Coding

**5.4** Construct code that uses JavaScript APIs

**5.4.1** Google Charts



# jQuery

jQuery is a JavaScript library meant to make JavaScript easier to write while also requiring less code. jQuery can be accessed by linking a script file or accessing the library through an API. For the exam, focus on jQuery's general syntax and purpose rather than learning specific jQuery methods.

## Purpose

Upon completing this project, you will better understand how to access the jQuery library.

## Steps for Completion

1. Open **jquery.html** from the Domain 5 Student folder.
2. On line 12, using jQuery, add a line of code that sets the inner HTML of the element with an id of heading to be equal to the string **Hello, World!**
3. On line 18, using jQuery, set the element's background color to lightgrey when the event listener fires.
4. Take a screenshot of lines 9-23.
5. Save the screenshot as **542-Completed**.

## Project Details

### Project file

jquery.html

### Estimated completion time

5 minutes

### Video reference

#### Domain 5

**Topic:** JavaScript APIs

**Subtopic:** jQuery

### Objectives covered

**5** JavaScript Coding

**5.4** Construct code that uses JavaScript APIs

**5.4.2** jQuery

# Geolocation

Websites use the Geolocation API to customize content based on a user's location. Possible use cases could include finding a restaurant, getting directions, or looking up local businesses. Whatever the case, knowing a user's general location is an important aspect of delivering content specific to that user.

## Purpose

Upon completing this project, you will better understand how to utilize the Geolocation API.

## Steps for Completion

1. Open **contact.html** and **location.js** from the Domain 5 Student folder.
2. On line 13, set the inner HTML of the element with the id of longitude to be equal to a message containing the user's longitude value.
3. Take a screenshot of lines 10-14.
4. Save the screenshot as **543a-Completed**.
5. Preview **contact.html** in a browser. Select the Get Location button. Open the console. Select the drop-down arrow for the Geolocation object. Take a screenshot of the Geolocation object.
6. Save the screenshot as **543b-Completed**.

## Project Details

### Project file

contact.html; location.js

### Estimated completion time

5 minutes

### Video reference

#### Domain 5

**Topic:** JavaScript APIs

**Subtopic:** Geolocation

### Objectives covered

**5** JavaScript Coding

**5.4** Construct code that uses JavaScript APIs

**5.4.3** Geolocation

# Domain 5

# Lesson 6

**HTML5**  
Application Development

# Fill-in-the-Blanks

**Instructions:** While watching Domain 5 Lesson 6, fill in the missing words according to the information presented by the instructor. [References are found in the brackets.]

1. Session state and application state typically describe state management methods for \_\_\_\_\_ applications. [Session State vs. App State]
2. The \_\_\_\_\_ method is used to add data to local storage or session storage. [Local vs. Session Storage]

# Local and Session Storage

Session state and application state describe two state management methods for ASP.NET applications. Application state refers to data that is available to all classes in an application, whereas session state stores data from previous requests.

Local and session storage allow developers to store key/value pairs in a browser. Upon closing a tab, all session storage is lost. Local storage has no expiration and remains in the browser until it is cleared. This storage can store information about a user and create a more personalized and immersive experience in future sessions.

## Purpose

Upon completing this project, you will better understand how to use local and session storage in a browser.

## Steps for Completion

1. \_\_\_\_\_ state typically stores requests for a pre-determined period.
2. The \_\_\_\_\_ state stores data in memory on a server and applies to all users and sessions for a particular server.
3. Open **contact.html** and **storage.js** from the Domain 5 Student folder.
4. On line 6, store the user\_firstName and userEmail variables as a key/value pair in the local storage.
5. Take a screenshot of line 6 of **storage.js**.
6. Save the screenshot as **552a-Completed**.
7. Preview **contact.html** in a browser. Enter a name and email into the bottom form. In the Developer Tools, find the local storage. Take a screenshot of the newly added key/value pair.
8. Save the screenshot as **552b-Completed**.

### Project Details

#### Project file

contact.html  
storage.js

#### Estimated completion time

15 minutes

#### Video reference

##### Domain 5

**Topic:** Application States

**Subtopic:** Session State vs. App State; Local vs. Session Storage

#### Objectives covered

##### 5 JavaScript Coding

**5.5** Manage the state of an application

**5.5.1** Session state vs. app state

**5.5.2** Where to store state (local vs. session storage)



# Appendix

**HTML5**  
Application Development



# Glossary

| Term                             | Definition                                                                                                                                                                  |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>.png</b>                      | An image file extension for Portable Network Graphics. This type of file is required for proper use of the drop-shadow CSS filter function.                                 |
| <b>.woff</b>                     | A web font format developed by Mozilla. Files with the .woff extension can be used to style webpages.                                                                       |
| <b>@keyframes</b>                | A CSS rule specifying characteristics of a CSS animation.                                                                                                                   |
| <b>3D Transformations</b>        | Three-dimensional effects on elements, including but not limited to rotate, translate, and scale.                                                                           |
| <b>Absolute Positioning</b>      | A technique for positioning elements in relation to their parent containers.                                                                                                |
| <b>addEventListener()</b>        | A method that fires a function or block of code when a specified event occurs.                                                                                              |
| <b>AJAX</b>                      | Asynchronous JavaScript and XML (AJAX) is the concept of using the XMLHttpRequest object to retrieve and process data without disrupting what has been loaded on a webpage. |
| <b>Animation</b>                 | The method of creating a sequence of movement on a screen.                                                                                                                  |
| <b>Animation-iteration-count</b> | A CSS property specifying the number of iterations for a CSS animation.                                                                                                     |
| <b>Animation-name</b>            | A CSS property specifying the name of an animation, linking it to an @keyframes rule set.                                                                                   |
| <b>Anonymous Function</b>        | A function without a name that can only be called by the variable where it is stored.                                                                                       |
| <b>API</b>                       | Application Programming Interface (API) is used to program graphical user interface (GUI) elements; includes a set of routines, protocols, and tools for building apps.     |
| <b>App Cache</b>                 | Application (App) Cache selectively saves files listed in a text file called a cache manifest.                                                                              |
| <b>App State</b>                 | A state that begins when a web browser sends the first request for a webpage to a web server and closes when the browser is closed.                                         |
| <b>Application</b>               | An application (app) is a program designed specifically for users.                                                                                                          |
| <b>Application Life Cycle</b>    | The complete process of planning, designing, writing, testing, packaging, validating, and deploying an app.                                                                 |
| <b>Background-image</b>          | A CSS property that adds a background image to an element.                                                                                                                  |
| <b>Background-size</b>           | A CSS property specifying the size of background images.                                                                                                                    |
| <b>Background-size: auto</b>     | A CSS property-value for the background-size property specifying that an image is displayed at its original size.                                                           |
| <b>Backticks</b>                 | A backtick ( ` ) is a symbol used to initialize template strings in JavaScript. It should not be confused with single quotes.                                               |
| <b>beginPath()</b>               | A built-in canvas method that begins a path or resets a current path.                                                                                                       |
| <b>Block</b>                     | A CSS property-value for the display property specifying that an element will take up the entire width of its parent container.                                             |
| <b>Blur</b>                      | A CSS filter function specifying a blur value for an image.                                                                                                                 |
| <b>Bootstrap</b>                 | A CSS framework known for creating responsive web designs that work well across mobile devices.                                                                             |
| <b>Border-box</b>                | A CSS property-value for the box-sizing property specifying that an element's border is contained within its overall size.                                                  |
| <b>Border-radius</b>             | A CSS property that rounds the corners of an element's border.                                                                                                              |
| <b>Box-Shadow</b>                | A CSS property that adds a shadow effect to the outside frame of an element.                                                                                                |
| <b>Box-sizing</b>                | A CSS property specifying how the web browser calculates an element's height and width regarding its border.                                                                |
| <b>Breakpoints</b>               | A designated stopping point in a program allowing a developer to analyze an application during execution.                                                                   |

| Term                        | Definition                                                                                                                                                                                                         |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bubbling</b>             | A term used to describe the order that events execute when multiple elements have been programmed to handle the same event. Specifically, inner events are handled first before bubbling outwards through the DOM. |
| <b>Bug</b>                  | An issue with an application that causes it to vary from the developer's expectations and/or user experience.                                                                                                      |
| <b>Cache</b>                | The act of storing resources on a client machine with the intent of faster load times.                                                                                                                             |
| <b>Callbacks</b>            | Executable code, usually in the form of a function, that is passed into a function as an argument.                                                                                                                 |
| <b>Canvas</b>               | An HTML element used to render graphics to a webpage.                                                                                                                                                              |
| <b>Canvas Tag</b>           | An HTML tag used to create a canvas element.                                                                                                                                                                       |
| <b>Cascading</b>            | A term used to describe the order by which events execute when multiple elements have been programmed to handle the same event. Specifically, outer events are handled before cascading inwards through the DOM.   |
| <b>Class, in JavaScript</b> | A template that is used to create JavaScript objects.                                                                                                                                                              |
| <b>clearRect()</b>          | A built-in canvas method that clears a designated rectangle within an HTML canvas.                                                                                                                                 |
| <b>Code Execution</b>       | The phase where a script is executed one line at a time. This phase runs separately from the memory allocation phase.                                                                                              |
| <b>Color Stop Position</b>  | Stopping points used to render a smooth color transition throughout a gradient.                                                                                                                                    |
| <b>Column-gap</b>           | A CSS property specifying the space between columns.                                                                                                                                                               |
| <b>Comment</b>              | A line of code that is ignored by a program. Comments are typically used to organize and explain code blocks.                                                                                                      |
| <b>Console</b>              | A section within the Developer Tools displaying error messages that allow developers to interact with and manipulate the current webpage using JavaScript.                                                         |
| <b>Constructor Function</b> | A function that is used to create an object.                                                                                                                                                                       |
| <b>Contain</b>              | A CSS property-value for the background-size property specifying that an entire image will be visible inside a container.                                                                                          |
| <b>Content-box</b>          | A CSS property-value for the box-sizing property specifying that an element's border is not contained within its overall size.                                                                                     |
| <b>Cover</b>                | A CSS property-value for the background-size property specifying that an image will resize to cover an entire container.                                                                                           |
| <b>createObjectURL()</b>    | A method used to return an object URL as a string.                                                                                                                                                                 |
| <b>Cross-Site Scripting</b> | A cyberattack where malicious code is inserted into a webpage to access data.                                                                                                                                      |
| <b>CSS</b>                  | Cascading Style Sheets (CSS) is a style sheet language used to add design and positioning properties to HTML elements on a webpage.                                                                                |
| <b>CSS Animations</b>       | A collection of CSS properties and rule sets allowing developers to animate HTML elements without JavaScript.                                                                                                      |
| <b>CSS Filter Function</b>  | A value that is added to the CSS Filter Property. For example, grayscale(); blur(); drop-shadow();.                                                                                                                |
| <b>CSS Filters</b>          | A CSS property specifying effects on an image, including blur, grayscale, and sepia.                                                                                                                               |
| <b>CSS Framework</b>        | A library with predefined CSS classes intended to make web design more simple and consistent.                                                                                                                      |
| <b>CSS Grid</b>             | A CSS module capable of creating flexible two-dimensional page layouts.                                                                                                                                            |
| <b>CSS Properties</b>       | Property/value combinations used to transform, style, and enhance text and graphics.                                                                                                                               |
| <b>CSS Reset</b>            | A boilerplate set of CSS rules at the top of a stylesheet designed to address inconsistencies across web browsers.                                                                                                 |
| <b>CSS Rule Set</b>         | The combination of a CSS selector, CSS properties, and property values.                                                                                                                                            |
| <b>Database</b>             | An organized collection of data, often featuring tables, queries, and reports.                                                                                                                                     |
| <b>Datalist</b>             | An HTML tag that creates a drop-down menu for an input field. Users are not confined to the options in a datalist.                                                                                                 |
| <b>Debugger</b>             | A keyword that can be added to HTML code to designate a breakpoint or stopping point.                                                                                                                              |

| Term                          | Definition                                                                                                                               |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Degrees</b>                | Degrees (deg) is a unit of measure used to define an angle.                                                                              |
| <b>Deployment</b>             | The process of initializing software support or integrating updated software into a current system.                                      |
| <b>Developer Tools</b>        | A collection of tools in a web browser used to analyze web applications.                                                                 |
| <b>Display</b>                | A CSS property-value combination specifying that an element should be removed from the flow of a page and not displayed.                 |
| <b>Display: flex</b>          | A CSS property-value combination that enables a container to function as a flexbox.                                                      |
| <b>Display: none</b>          | A CSS property-value combination specifying that an element be removed from the flow of a page and not displayed.                        |
| <b>DOM</b>                    | A Document Object Model (DOM) is a standard structure for an HTML document. JavaScript can be used to modify all elements of the DOM.    |
| <b>Downtime</b>               | The length of time a system is shut down due to maintenance or technical issues.                                                         |
| <b>Drop-shadow</b>            | A CSS filter function that adds a shadow to the shape of an image rather than its natural rectangular border.                            |
| <b>Dynamic Content</b>        | Content that changes in real-time based on a user and their actions.                                                                     |
| <b>Empty String</b>           | A string with no characters. In JavaScript, a variable set to an empty string can be manipulated as a string.                            |
| <b>Equals Operator</b>        | A single equal sign specifying the value of a variable or property. A double equal sign checks for equality and returns a Boolean value. |
| <b>Eric Meyer's CSS Reset</b> | A popular CSS reset for addressing inconsistencies between web browsers.                                                                 |
| <b>ES6</b>                    | A major update to JavaScript in 2015 that added significant new features.                                                                |
| <b>Event</b>                  | An action that occurs throughout the lifecycle of a program initiated by either an end-user or the program itself.                       |
| <b>Event Listener</b>         | A built-in JavaScript method used to execute code when a specific action is taken on a specific element.                                 |
| <b>Event Object</b>           | A JavaScript object that is created as a result of a successful event listener.                                                          |
| <b>Extend</b>                 | A JavaScript keyword used along with the class keyword to create a new class by inheriting another class.                                |
| <b>Extension</b>              | Specific functionality that can be added onto a larger existing program, such as a web browser or code editor.                           |
| <b>External Files</b>         | Files from outside the current project or folder.                                                                                        |
| <b>feGaussianBlur</b>         | An SVG filter that blurs an element.                                                                                                     |
| <b>Fetch API</b>              | Modern technique used by web browsers to make HTTP requests.                                                                             |
| <b>Fieldset</b>               | An HTML tag that groups HTML form elements and places a box around those elements.                                                       |
| <b>FileReader Object</b>      | A built-in object that is used by a web browser to read files asynchronously.                                                            |
| <b>fillRect()</b>             | A built-in canvas method that draws a filled rectangle onto an HTML canvas.                                                              |
| <b>Filter Attribute</b>       | An attribute added to an SVG element that links the element to a predefined SVG filter.                                                  |
| <b>Flex</b>                   | A CSS property that is shorthand for the flex-grow, flex-shrink, and flex-basis properties.                                              |
| <b>Flex-basis</b>             | A CSS property specifying the initial size of a flex item along the flex container main axis.                                            |
| <b>Flexbox</b>                | A CSS module capable of creating flexible one-dimensional page layouts.                                                                  |
| <b>Flex-direction</b>         | A CSS property specifying how a flexbox will display multiple items.                                                                     |
| <b>Flex-flow</b>              | A CSS property specifying whether items will wrap inside of a flex container.                                                            |
| <b>Flex-grow</b>              | A CSS property specifying how a flex item will grow inside of a flex container.                                                          |
| <b>Flex-shrink</b>            | A CSS property specifying the how a flex item will shrink inside of a flex container.                                                    |
| <b>Flex-wrap</b>              | A CSS property that is shorthand for the flex-direction and flex-wrap properties.                                                        |
| <b>Float</b>                  | A CSS property used to remove an element from the natural flow of a webpage and shift that element to the left or right.                 |

| Term                         | Definition                                                                                                                                                            |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Font-family</b>           | A CSS property specifying the font for an element.                                                                                                                    |
| <b>Font-style</b>            | A CSS property that is used to display an italic font style.                                                                                                          |
| <b>Font-weight</b>           | A CSS property that is used to display a bold or thin font style.                                                                                                     |
| <b>For-each Loop</b>         | A programming logic component where a code block is run once for every element in an array or list.                                                                   |
| <b>Forms</b>                 | A section of a webpage designed to accept input from users.                                                                                                           |
| <b>Fractional Units</b>      | Fractional (fr) units are relative CSS units that size elements based on adjacent fractional values.                                                                  |
| <b>Gap</b>                   | A CSS property that is shorthand for the row-gap and column-gap properties.                                                                                           |
| <b>Geolocation API</b>       | Web browser functionality that enables programs to request and then use an end-user's location.                                                                       |
| <b>GetCurrentPosition()</b>  | A method available to a Geolocation Object that retrieves the location of a device.                                                                                   |
| <b>Git</b>                   | Software used to track changes to a collection of files.                                                                                                              |
| <b>GitHub</b>                | A code hosting platform for collaboration and version control.                                                                                                        |
| <b>Global Variable</b>       | A variable that can be accessed from all sections of a program.                                                                                                       |
| <b>Google Fonts</b>          | An online font library featuring free and open-source font families. Fonts are accessed through the Google Fonts API.                                                 |
| <b>Gradient</b>              | A smooth transition from one color to another.                                                                                                                        |
| <b>Graphic Elements</b>      | Visual elements that make up a webpage.                                                                                                                               |
| <b>Grayscale</b>             | A CSS filter function that gradually converts an image to a colorless version.                                                                                        |
| <b>Grid Gap</b>              | Space between rows and columns in a grid container.                                                                                                                   |
| <b>Grid Line</b>             | A reference to the individual lines that divide the rows and columns of a grid. Lines are 1-based.                                                                    |
| <b>Grid-area</b>             | A CSS property specifying a variable name for an element. This value is used to organize a grid layout within the grid-template property.                             |
| <b>Grid-Based Layouts</b>    | Webpage layouts based on several columns that make up 100% of the page width.                                                                                         |
| <b>Grid-column</b>           | A CSS property that is shorthand for the grid-column-start and grid-column-end properties.                                                                            |
| <b>Grid-template</b>         | A CSS property specifying how predefined grid-areas should be arranged in a grid container.                                                                           |
| <b>Grid-template-columns</b> | A CSS property specifying how CSS Grids display columns.                                                                                                              |
| <b>Grid-template-rows</b>    | A CSS property specifying how CSS Grids display rows.                                                                                                                 |
| <b>Grid-view</b>             | A grid-view divides webpages up into several columns (usually 12) that enable a developer to control page layout using predefined CSS classes based on those columns. |
| <b>Hexadecimal</b>           | A base-16 number type representing numbers with digits from 0 through 9 and letters from A through F.                                                                 |
| <b>Hidden</b>                | A CSS property-value for the overflow property specifying that a browser should hide content that overflows an element.                                               |
| <b>HTML Attribute</b>        | A characteristic of an HTML element that can be customized statically or dynamically.                                                                                 |
| <b>HTML Element</b>          | An element defined by HTML tags, used as part of a webpage.                                                                                                           |
| <b>html()</b>                | A jQuery function specifying the inner HTML of an element.                                                                                                            |
| <b>HTMLCollection</b>        | A collection of HTML elements similar to an array.                                                                                                                    |
| <b>If/else statement</b>     | A JavaScript statement that evaluates an expression and different blocks of code based on whether the expression is true or false.                                    |
| <b>Image Tag</b>             | An HTML tag used to define an image on a webpage.                                                                                                                     |
| <b>Inheritance</b>           | The programming concept where a newly defined class can assume the properties and methods of a previously defined class.                                              |
| <b>Initial-scale</b>         | A CSS viewport meta tag property specifying that a webpage should not zoom during the initial loading process.                                                        |
| <b>Injection Attack</b>      | A category of cyberattack where the attacker inputs malicious code into a form or database, causing an application to function differently.                           |

| Term                        | Definition                                                                                                                                                                                |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Inline</b>               | A CSS property-value for the display property specifying that an element can display alongside other inline elements.                                                                     |
| <b>Inline XML</b>           | XML code that is written directly into an HTML file rather than imported using a filename.                                                                                                |
| <b>Inline-block</b>         | A CSS property-value for the display property specifying that an element can display inline with other elements while also accepting width and height values.                             |
| <b>InnerHTML</b>            | The property of an HTML element that contains the characters between the opening and closing tag of the element.                                                                          |
| <b>Input Validation</b>     | The act of ensuring input data matches an expected format.                                                                                                                                |
| <b>Instantiation</b>        | The creation of an object based on a template or class.                                                                                                                                   |
| <b>Internal Stylesheet</b>  | A CSS stylesheet that is added to the head section of an HTML document.                                                                                                                   |
| <b>JavaScript</b>           | A scripting language used to create dynamic web content.                                                                                                                                  |
| <b>JavaScript Functions</b> | Reusable JavaScript code that performs actions on data.                                                                                                                                   |
| <b>jQuery</b>               | A JavaScript library designed to make JavaScript easier to write while requiring less code.                                                                                               |
| <b>JSON</b>                 | JavaScript Object Notation (JSON) is an open-standard file format that uses human-readable text to transmit data consisting of key/value pairs.                                           |
| <b>JSON String</b>          | A JavaScript string that can be parsed into a JavaScript object.                                                                                                                          |
| <b>JSON.parse()</b>         | A built-in JavaScript method that takes a JSON string as a parameter and returns a JavaScript object.                                                                                     |
| <b>Justify-content</b>      | A CSS property specifying how CSS Flexbox spaces items inside of a flex container.                                                                                                        |
| <b>Key/Value Pairs</b>      | A technique where data can be stored and retrieved using a unique key.                                                                                                                    |
| <b>Keyframes</b>            | Specific points throughout a CSS animation.                                                                                                                                               |
| <b>Legend</b>               | An HTML tag specifying a title for a form or fieldset.                                                                                                                                    |
| <b>Letter-spacing</b>       | A CSS property that is used to specify the horizontal space between characters in a word.                                                                                                 |
| <b>Linear Gradient</b>      | A smooth transition from one color to another along a straight axis.                                                                                                                      |
| <b>LineTo()</b>             | A built-in canvas method that designates a new point and draws a straight line from the previous point on the canvas to the new point.                                                    |
| <b>Local Storage</b>        | Browser-enabled storage that provides the ability to store key/value pairs. Local storage is constant unless explicitly cleared or deleted.                                               |
| <b>Logic Errors</b>         | An error in the flow of an application that typically does not cause an error message.                                                                                                    |
| <b>Loops</b>                | A programming logic component where an action or group of actions is run either a set number of times or until a condition is met.                                                        |
| <b>Lorem Ipsum</b>          | Placeholder text frequently used in design layouts.                                                                                                                                       |
| <b>Malicious Attack</b>     | An attack designed to disrupt operations and steal sensitive information.                                                                                                                 |
| <b>Margin</b>               | A CSS property specifying space around the outside of elements.                                                                                                                           |
| <b>Max-length</b>           | An HTML attribute specifying the maximum number of characters allowed in an input tag or textarea tag.                                                                                    |
| <b>MDN Web Docs</b>         | An open-source collection of documentation for web development.                                                                                                                           |
| <b>Media Queries</b>        | A CSS3 technique that evaluates an expression to ensure that appropriate classes are applied to elements (normally based on screen size), with the main goal being responsive web design. |
| <b>Meta Tag</b>             | An HTML tag that specifies the overall characteristics of an HTML page. Meta tags are placed inside the head section of an HTML page.                                                     |
| <b>Meter</b>                | An HTML tag that creates a visual of a measurement or fractional value.                                                                                                                   |
| <b>Method</b>               | In programming, an action that takes place, such as changing a property's attribute.                                                                                                      |
| <b>Microsoft Edge</b>       | Popular modern-day web browser.                                                                                                                                                           |
| <b>Mouse Event</b>          | An event that returns an event object containing properties that describe the state of the mouse.                                                                                         |
| <b>Mouseout (event)</b>     | A JavaScript event that fires when the mouse pointer leaves an element.                                                                                                                   |
| <b>Mouseover (event)</b>    | A JavaScript event that fires when the mouse pointer hovers on an element.                                                                                                                |
| <b>MoveTo()</b>             | A built-in canvas method used to move to a specified point on a canvas without drawing a line.                                                                                            |

| Term                         | Definition                                                                                                                                                                                                                                                                                                             |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>OffsetX</b>               | A property of a Mouse Event that describes the x-coordinate of the pointer related to the target element.                                                                                                                                                                                                              |
| <b>OffsetY</b>               | A property of a Mouse Event that describes the y-coordinate of the pointer related to the target element.                                                                                                                                                                                                              |
| <b>OOP</b>                   | Object-Oriented Programmed (OOP) is a style of programming that is designed around objects and their properties that can contain data and code.                                                                                                                                                                        |
| <b>Opacity</b>               | A CSS property that specifies the transparency level for an element. Opacity can also be used to style an image using the CSS filter property along with the Opacity CSS Filter Function.                                                                                                                              |
| <b>Output</b>                | An HTML tag that displays the results of a calculation or script.                                                                                                                                                                                                                                                      |
| <b>Overflow</b>              | A CSS property specifying how a browser should react when content overflows within an element.                                                                                                                                                                                                                         |
| <b>Pattern Attribute</b>     | An HTML attribute that can validate an input field using regular expressions.                                                                                                                                                                                                                                          |
| <b>Photoshop</b>             | A photo editing and graphic design tool used to create and edit graphics and digital images.                                                                                                                                                                                                                           |
| <b>Pixel</b>                 | A pixel (px) is a unit of measure used to define the smallest unit in a graphic display or digital image.                                                                                                                                                                                                              |
| <b>Pixelation</b>            | The effect of adjusting an image such that the human eye easily sees individual pixels.                                                                                                                                                                                                                                |
| <b>Preview on Web Server</b> | An extension for Visual Studio Code used to render a real-time preview of a webpage to a browser.                                                                                                                                                                                                                      |
| <b>Properties</b>            | In JavaScript, named or numbered values that make up a JavaScript object.                                                                                                                                                                                                                                              |
| <b>Radial Gradient</b>       | A smooth transition from one color to another in a circle or oval shape.                                                                                                                                                                                                                                               |
| <b>ReadyState</b>            | A property of a request that describes the current loading state of a request.                                                                                                                                                                                                                                         |
| <b>Readystatechange</b>      | A JavaScript event that fires when there is a change to the readyState of a request.                                                                                                                                                                                                                                   |
| <b>Referenced XML</b>        | XML code that is imported into a file using a hyperlink or file location.                                                                                                                                                                                                                                              |
| <b>Regular Expressions</b>   | A sequence of characters used to describe a string pattern. That pattern is then used to search and replace strings or portions of strings.                                                                                                                                                                            |
| <b>Relative Positioning</b>  | A technique for positioning elements in relation to their natural position within a webpage's flow.                                                                                                                                                                                                                    |
| <b>Render</b>                | The act of drawing content to a browser window.                                                                                                                                                                                                                                                                        |
| <b>Required</b>              | An HTML attribute used to throw an error during form submission if a form field is empty.                                                                                                                                                                                                                              |
| <b>ResponseText</b>          | A property of the XMLHttpRequest object that receives text from a successful server request.                                                                                                                                                                                                                           |
| <b>ResponseXML</b>           | A property of the XMLHttpRequest object that receives XML data from a server request.                                                                                                                                                                                                                                  |
| <b>Responsive Web Design</b> | A design concept that uses various techniques to ensure that a webpage has an optimal design for various screens and devices.                                                                                                                                                                                          |
| <b>Responsive Width</b>      | A characteristic of an HTML element specifying that an element's width is calculated based on the size of the browser window or parent container.                                                                                                                                                                      |
| <b>Reusable Code</b>         | Code that is written once and used in more than one instance.                                                                                                                                                                                                                                                          |
| <b>Rotate</b>                | 1: A built-in canvas method that rotates a canvas on the (0,0) point by a certain number of degrees. This method will only affect canvas elements drawn after this method is called. 2: A CSS property value that rotates an element by a specified degree value when used in conjunction with the transform property. |
| <b>Row-gap</b>               | A CSS property specifying the space between rows for a grid.                                                                                                                                                                                                                                                           |
| <b>Runtime</b>               | The period of time when a program is actively running.                                                                                                                                                                                                                                                                 |
| <b>Sandbox</b>               | An isolated environment used to test software.                                                                                                                                                                                                                                                                         |
| <b>Saturate</b>              | A CSS filter function that over-saturates an image.                                                                                                                                                                                                                                                                    |
| <b>Scale</b>                 | A CSS property value that, when used in conjunction with the transform property, scales the size of an element either up or down.                                                                                                                                                                                      |
| <b>Scroll</b>                | A CSS property-value for the overflow property specifying that a browser should add a scroll bar if content overflows an element.                                                                                                                                                                                      |
| <b>Send()</b>                | A built-in method of the XMLHttpRequest object that sends a request to a server.                                                                                                                                                                                                                                       |



| Term                              | Definition                                                                                                                                                                          |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sepia</b>                      | A CSS filter function that gradually converts an image to a brownish-red color, usually associated with old-style photographs.                                                      |
| <b>Session State</b>              | An ASP.NET state that allows a specific user to store data on a web server and call that data anytime during a session.                                                             |
| <b>Session Storage</b>            | Browser-enabled storage that provides the ability to store key/value pairs. Session storage is deleted when the browser tab is closed.                                              |
| <b>Skew</b>                       | A CSS property value that, when used in conjunction with the transform property, tilts an image.                                                                                    |
| <b>SQL Injection</b>              | A cyberattack where malicious code is inserted into a database with the intent of reading or modifying data.                                                                        |
| <b>String Literal</b>             | Also referred to as Template Strings, these strings are declared in JavaScript using backticks and allow the developer to inject expressions and JavaScript (interpolation).        |
| <b>Stroke()</b>                   | A built-in canvas method that draws a line after two points have been designated using the moveTo method and lineTo method.                                                         |
| <b>StrokeRect()</b>               | A built-in canvas method that draws an unfilled rectangle to an HTML canvas.                                                                                                        |
| <b>StrokeStyle</b>                | An HTML property used to set the color for the outline of a path.                                                                                                                   |
| <b>Style Attribute</b>            | An attribute added to an HTML element that is used to set styles for an element.                                                                                                    |
| <b>SVG</b>                        | A scalable vector graphic (SVG) is a two-dimensional graphic created by XML-style markup.                                                                                           |
| <b>SVG Element</b>                | A container that defines a new coordinate system within an SVG document.                                                                                                            |
| <b>SVG Tag</b>                    | An HTML tag that contains an SVG element.                                                                                                                                           |
| <b>Syntax Error</b>               | An error in an application's source code, most commonly a spelling error or punctuation error. These errors are typically underlined with a red squiggly line in most code editors. |
| <b>Tailwind CSS</b>               | A CSS framework, made unique by its almost exclusive use of class names to design elements.                                                                                         |
| <b>Target Element</b>             | The element that triggers a given event.                                                                                                                                            |
| <b>Textarea</b>                   | An HTML tag that creates a large input field for users.                                                                                                                             |
| <b>textContent</b>                | The property of an HTML element that returns a string containing the text from each child of that element.                                                                          |
| <b>Text-Shadow</b>                | A CSS property that adds a shadow effect to text.                                                                                                                                   |
| <b>This</b>                       | A JavaScript keyword generally used to refer to an object itself, although use cases vary.                                                                                          |
| <b>ToFixed()</b>                  | A JavaScript function that formats a type Number to a set number of decimal places.                                                                                                 |
| <b>Transformations</b>            | Two-dimensional or three-dimensional effects on elements using CSS, including but not limited to rotate, scale, skew, and translate.                                                |
| <b>Transition</b>                 | A CSS property that is used to specify the transitioning property and duration of the transition.                                                                                   |
| <b>Transition-delay</b>           | A CSS property that is used to delay a previously defined transition.                                                                                                               |
| <b>Transition-timing-function</b> | A CSS property that is used to specify the speed throughout a transition.                                                                                                           |
| <b>Translate</b>                  | A built-in canvas method that designates a new position on the canvas as (0,0).                                                                                                     |
| <b>Type</b>                       | An HTML attribute specifying the type for an HTML input tag.                                                                                                                        |
| <b>UI</b>                         | A user interface (UI) is the way that a user and a computer system interact.                                                                                                        |
| <b>UseCapture</b>                 | The third parameter for the addEventListener method that takes a Boolean value, specifying whether or not events for an element are set to bubble or cascade.                       |
| <b>User-Initiated Event</b>       | An event that fires due to the actions of an end-user.                                                                                                                              |
| <b>Variable</b>                   | A container that holds data, with the intent that the program will use the data many times.                                                                                         |
| <b>Viewheight</b>                 | Viewheight (vh) is a CSS unit of measurement specifying the height of an element relative to the viewport's height.                                                                 |
| <b>Viewport</b>                   | Viewport is typically used in the context of a meta tag, where it specifies that the content width is set equal to the device width.                                                |
| <b>Viewwidth</b>                  | Viewwidth (vw) is a CSS unit of measurement specifying the width of an element relative to the viewport's width.                                                                    |

| Term                         | Definition                                                                                                                                                              |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Visual Studio Code</b>    | A free, open-source integrated development environment created by Microsoft.                                                                                            |
| <b>Web Server</b>            | Software that distributes and displays web content in a browser by accepting HTTP requests.                                                                             |
| <b>Web-Safe Fonts</b>        | Fonts that are readily available on almost any device or browser. These fonts are used alongside imported fonts, ensuring that at least one declared font is available. |
| <b>Width</b>                 | A CSS property specifying a width for a given element.                                                                                                                  |
| <b>Window Event</b>          | An event that returns an event object containing properties that describe the state of the browser window.                                                              |
| <b>XML</b>                   | Extensible Markup Language (XML) is a markup language, similar to HTML, designed to store and transport data.                                                           |
| <b>XMLHttpRequest Object</b> | A built-in JavaScript object that manages the full lifecycle of a request to an API or web server.                                                                      |

# Objectives

| HTML5 Application Development Objectives                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Domain 1<br>Application Lifecycle Management                                                                        | Domain 2<br>Graphics and Animation                                                                                                                                                                                                                                                                                                                                                                                                                           | Domain 3<br>Forms                                                                                                                                                                   | Domain 4<br>Layouts                                                                                                                                                                                                                                                            | Domain 5<br>JavaScript Coding                                                                                                                                                                    |
| 1.1 Describe the application lifecycle management stages<br>1.1.1 Plan, design, develop, test, deploy, and maintain | 2.1 Use the canvas element to create graphics and animations<br>2.1.1 Shape<br>2.1.2 Color<br>2.1.3 Line<br>2.1.4 Translation/move<br>2.1.5 Rotate<br>2.1.6 Scale<br>2.1.7 Interactivity                                                                                                                                                                                                                                                                     | 3.1 Construct and analyze markup that uses form elements<br>datalist, fieldset, meter, legend, output                                                                               | 4.1 Manage content layout, positioning, and flow by using CSS<br>4.1.1 Content flow (inline vs. block flow)<br>4.1.2 Positioning of individual elements (float vs. absolute positioning)<br>4.1.3 Content overflow (scrolling, visible, and hidden)<br>4.1.4 Basic CSS styling | 5.1 Create and use custom classes<br>5.1.1 Instantiation<br>5.1.2 Properties<br>5.1.3 Methods<br>5.1.4 Inheritance                                                                               |
| 1.2 Debug and test web apps<br>1.2.1 Input validation errors<br>1.2.2 Runtime errors<br>1.2.3 Breakpoints           | 2.2 Use the SVG element to create and display graphics<br>2.2.1 Advantages<br>2.2.2 Inline vs. referenced XML<br>2.2.3 Shapes<br>2.2.4 Color<br>2.2.5 SVG filter effects                                                                                                                                                                                                                                                                                     | 3.2 Configure input validation<br>3.2.1 Validation attributes<br>3.2.2 Pattern attribute for regular expressions<br>3.2.3 Correct data type<br>3.2.4 Length<br>3.2.5 Required value | 4.2 Construct layouts by using responsive design<br>4.2.1 Grid view<br>4.2.2 Background-size<br>4.2.3 Images<br>4.2.4 Picture<br>4.2.5 Viewport<br>4.2.6 Responsive width<br>4.2.7 Media queries                                                                               | 5.2 Perform data access by using JavaScript<br>5.2.1 Send and receive data<br>5.2.2 Transmit and parse complex objects<br>5.2.3 Load and save files<br>5.2.4 XML<br>5.2.5 JSON                   |
|                                                                                                                     | 2.3 Transform, style, and enhance text and graphics<br>2.3.1 Graphics effects (rounded corners, shadows, transparency, background gradients, typography, and Web Open Font Format)<br>2.3.1.1 Rounded corners, shadows, transparency<br>2.3.1.2 Background gradients<br>2.3.1.3 Typography and Web Open Font Format<br>2.3.2 2-D and 3-D transformations (translate, scale, rotate, skew, and 3-D perspective transitions and animations)<br>2.3.3 keyframes |                                                                                                                                                                                     | 4.3 Construct flexible responsive, layouts by using CSS flexbox<br>4.3.1 Flex container (flex-direction, flex-flow, flex-wrap)<br>4.3.2 Flex items (flex-basis, flex-grow, flex-shrink, order, flex)                                                                           | 5.3 Construct code that responds to events by using event listeners and handlers<br>5.3.1 Gesture events<br>5.3.2 Handling multiple events<br>5.3.3 Event object<br>5.3.4 Bubbling vs. cascading |
|                                                                                                                     | 2.4 Apply CSS filters to images<br>2.4.1 Grayscale<br>2.4.2 Blur<br>2.4.3 Sepia<br>2.4.4 Opacity<br>2.4.5 Drop shadow<br>2.4.6 Saturate                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                     | 4.4 Construct grid-based layouts by using CSS grid<br>4.4.1 Container<br>4.4.2 Items<br>4.4.3 Templates<br>4.4.4 Gap/Lines                                                                                                                                                     | 5.4 Construct code that uses JavaScript APIs<br>5.4.1 Google Charts<br>5.4.2 jQuery<br>5.4.3 Geolocation                                                                                         |

# Lesson Plan

Approximately 24 hours of video, labs, and projects

## HTML5

Application Development

# Domain 1 Lesson Plan

| Domain 1 - Application Lifecycle Management [approximately 2 hours of videos, labs, and projects]    |                                                                                                                                                                                  |                                                                                                                                                                                                                                  |                              |                                                                                              |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|----------------------------------------------------------------------------------------------|
| Lesson                                                                                               | Lesson Topic and Subtopics                                                                                                                                                       | Objectives                                                                                                                                                                                                                       | Exercise Labs                | Workbook Projects and Files                                                                  |
| <b>Pre-Assessment</b><br>Assessment time - 00:15:00                                                  | Application Lifecycle Management: Pre-Assessment                                                                                                                                 |                                                                                                                                                                                                                                  |                              |                                                                                              |
| <b>Lesson 1</b><br>Video time - 00:13:51<br>Exercise Lab time - 00:04:00<br>Workbook time - 00:25:00 | <b>Application Lifecycle Management</b><br>Introduction<br>How to Study for This Exam<br>Application Lifecycle Steps<br>Input Validation Errors<br>Runtime Errors<br>Breakpoints | 1.1 Describe the application lifecycle management stages<br>1.1.1 Plan, design, develop, test, deploy, and maintain<br>1.2 Debug and test web apps<br>1.2.1 Input validation errors<br>1.2.2 Runtime errors<br>1.2.3 Breakpoints | Understanding Runtime Errors | Application Lifecycle Steps – pg. 8<br>N/A<br>Debug and Test Web Applications – pg. 9<br>N/A |
| <b>Post-Assessment</b><br>Assessment time - 00:40:00                                                 | Application Lifecycle Management: Post-Assessment                                                                                                                                |                                                                                                                                                                                                                                  |                              |                                                                                              |



# Domain 2 Lesson Plan

| Domain 2 - Graphics and Animation [approximately 8 hours of videos, labs, and projects]              |                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lesson                                                                                               | Lesson Topic and Subtopics                                                                                                                                        | Objectives                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Exercise Labs                                                                                                                                                                                                                         | Workbook Projects and Files                                                                                                                                                                                                                                                                          |
| <b>Pre-Assessment</b><br>Assessment time - 00:40:00                                                  | Graphics and Animation: Pre-Assessment                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                      |
| <b>Lesson 1</b><br>Video time - 00:08:00<br>Exercise Lab time - 00:12:00<br>Workbook time - 00:40:00 | <b>Canvas Element Part 1</b><br>Shape<br>Color<br>Line                                                                                                            | 2.1 Use the canvas element to create graphics and animations<br>2.1.1 Shape<br>2.1.2 Color<br>2.1.3 Line                                                                                                                                                                                                                                                                                                                                                     | Code Block<br>Creating Shapes<br>Adding Color to Shapes                                                                                                                                                                               | Canvas: Shape – pg. 12 index.html; canvas.js<br>Canvas: Color – pg. 13 index.html; canvas.js<br>Canvas: Line – pg. 14 index.html; canvas.js                                                                                                                                                          |
| <b>Lesson 2</b><br>Video time - 00:08:23<br>Exercise Lab time - 00:16:00<br>Workbook time - 00:40:00 | <b>Canvas Element Part 2</b><br>Translation<br>Rotate<br>Scale<br>Interactivity                                                                                   | 2.1.4 Translation/move<br>2.1.5 Rotate<br>2.1.6 Scale<br>2.1.7 Interactivity                                                                                                                                                                                                                                                                                                                                                                                 | Translate Method<br>Rotating a Circle<br>Understanding the Scale Method<br>Understanding Interactivity                                                                                                                                | Canvas: Translation – pg. 17 index.html; canvas1.js<br>Canvas: Rotate – pg. 18 index.html; canvas1.js<br>Canvas: Scale – pg. 19 index.html; canvas1.js<br>Canvas: Interactivity – pg. 20 index.html; coords.js                                                                                       |
| <b>Lesson 3</b><br>Video time - 00:09:32<br>Exercise Lab time - 00:12:00<br>Workbook time - 00:35:00 | <b>SVG Element</b><br>Advantages<br>Inline vs. Referenced<br>XML<br>Shapes<br>Color<br>SVG Filter Effects                                                         | 2.2 Use the SVG element to create and display graphics<br>2.2.1 Advantages<br>2.2.2 Inline vs. referenced XML<br>2.2.3 Shapes<br>2.2.4 Color<br>2.2.5 SVG filter effects                                                                                                                                                                                                                                                                                     | Understanding the Rendering of SVGs<br>Further<br>Understanding the Rendering of SVGs<br>Adding a Filter                                                                                                                              | SVG: Scalable Vector Graphics – pg. 23 about.html<br>SVG: Shapes and Color – pg. 24 about1.html<br>SVG: Filter Effects – pg. 25 about.html                                                                                                                                                           |
| <b>Lesson 4</b><br>Video time - 00:13:31<br>Exercise Lab time - 00:32:00<br>Workbook time - 01:00:00 | <b>Graphics Effects</b><br>Rounded Corners, Shadows, and Transparency<br>Background Gradients<br>Typography and WOFF<br>2D and 3D<br>Transformations<br>Keyframes | 2.3 Transform, style, and enhance text and graphics<br>2.3.1 Graphics effects (rounded corners, shadows, transparency, background gradients, typography, and Web Open Font Format)<br>2.3.1.1 Rounded corners, shadows, transparency<br>2.3.1.2 Background gradients<br>2.3.1.3 Typography and Web Open Font Format<br>2.3.2 2-D and 3-D transformations (translate, scale, rotate, skew, and 3-D perspective transitions and animations)<br>2.3.3 keyframes | Further<br>Understanding CSS Properties<br>Further<br>Comprehending CSS Properties<br>Gradient Types<br>Other Gradient Types<br>Gradient Directions<br>Adding Fonts<br>Understanding<br>Keyframe Syntax<br>Understanding<br>Keyframes | Rounded Corners, Shadows, and Transparency – pg. 28 about.html; main.css<br>Background Gradients – pg. 29 full-sized-div.html<br>Typography and Web Open Font Format – pg. 30 about.html; main.css<br>2D and 3D<br>Transformations– pg. 31 transformations.html<br>Keyframes – pg. 32 keyframes.html |
| <b>Lesson 5</b><br>Video time -                                                                      | <b>CSS Filters on Images</b><br>Grayscale                                                                                                                         | 2.4 Apply CSS filters to images<br>2.4.1 Grayscale                                                                                                                                                                                                                                                                                                                                                                                                           | Blur<br>Shadows                                                                                                                                                                                                                       | CSS Image Filters – pg. 33 image-filters.html                                                                                                                                                                                                                                                        |



## Domain 2 - Graphics and Animation [approximately 8 hours of videos, labs, and projects]

| Lesson                                                                           | Lesson Topic and Subtopics                          | Objectives                                                                        | Exercise Labs | Workbook Projects and Files                    |
|----------------------------------------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------------|---------------|------------------------------------------------|
| 00:07:37<br>Exercise Lab<br>time -<br>00:08:00<br>Workbook<br>time -<br>00:30:00 | Blur<br>Sepia<br>Opacity<br>Drop-Shadow<br>Saturate | 2.4.2 Blur<br>2.4.3 Sepia<br>2.4.4 Opacity<br>2.4.5 Drop shadow<br>2.4.6 Saturate |               | CSS Drop Shadow – pg. 36<br>image-filters.html |
| <b>Post-Assessment</b><br>Assessment<br>time -<br>01:30:00                       | Graphics and Animation: Post-Assessment             |                                                                                   |               |                                                |

# Domain 3 Lesson Plan

| Domain 3 - Forms [approximately 1.5 hours of videos, labs, and projects]                             |                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                    |                                     |                                                                                     |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------|
| Lesson                                                                                               | Lesson Topic and Subtopics                                                                                                                                                             | Objectives                                                                                                                                                                                                                                                                                         | Exercise Labs                       | Workbook Projects and Files                                                         |
| <b>Pre-Assessment</b><br>Assessment time - 00:15:00                                                  | Forms: Pre-Assessment                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                    |                                     |                                                                                     |
| <b>Lesson 1</b><br>Video time - 00:11:54<br>Exercise Lab time - 00:04:00<br>Workbook time - 00:40:00 | <b>Form Markup</b><br>Form Elements<br><b>Configure Input</b><br><b>Validation</b><br>Validation<br>Attributes<br>Pattern Attributes<br>Correct Data Types<br>Length<br>Required Value | 3.1 Construct and analyze markup that uses form elements<br>3.1.1 Datalist, fieldset, meter, legend, output<br>3.2 Configure input validation<br>3.2.1 Validation attributes<br>3.2.2 Pattern attribute for regular expressions<br>3.2.3 Correct data type<br>3.2.4 Length<br>3.2.5 Required value | Understanding the Pattern Attribute | Form Elements – pg. 39<br>contact.html<br>Input Validation – pg. 40<br>contact.html |
| <b>Post-Assessment</b><br>Assessment time - 00:30:00                                                 | Forms: Post-Assessment                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                    |                                     |                                                                                     |

# Domain 4 Lesson Plan

| Domain 4 - Layouts [approximately 6 hours of videos, labs, and projects]                             |                                                                                                                                            |                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                 |                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lesson                                                                                               | Lesson Topic and Subtopics                                                                                                                 | Objectives                                                                                                                                                                                                                                                                     | Exercise Labs                                                                                                                                                                                   | Workbook Projects and Files                                                                                                                                      |
| <b>Pre-Assessment</b><br>Assessment time - 00:30:00                                                  | Layouts: Pre-Assessment                                                                                                                    |                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                 |                                                                                                                                                                  |
| <b>Lesson 1</b><br>Video time - 00:11:55<br>Exercise Lab time - 00:28:00<br>Workbook time - 00:30:00 | <b>Layout, Positioning, and Flow</b><br>Content Flow<br>Element<br>Positioning<br>Content Overflow<br>Basic CSS Styling                    | 4.1 Manage content layout, positioning, and flow by using CSS<br>4.1.1 Content flow (inline vs. block flow)<br>4.1.2 Positioning of individual elements (float vs. absolute positioning)<br>4.1.3 Content overflow (scrolling, visible, and hidden)<br>4.1.4 Basic CSS styling | Display Layouts<br>More Display Layouts<br>Other Display Layouts<br>Positioning of Elements<br>More Positioning of Elements<br>Further Positioning of Elements<br>Other Positioning of Elements | Content Flow – pg. 43<br>flow.html<br>Positioning – pg. 44<br>positioning.html<br>Content Overflow and CSS Reset – pg. 45<br>main.css                            |
| <b>Lesson 2</b><br>Video time - 00:08:50<br>Exercise Lab time - 00:04:00<br>Workbook time - 00:35:00 | <b>Responsive Design Part 1</b><br>Grid View<br>Background Size<br>Images and Pictures<br>Viewports                                        | 4.2 Construct layouts by using responsive design<br>4.2.1 Grid view<br>4.2.2 Background-size<br>4.2.3 Images<br>4.2.4 Picture<br>4.2.5 Viewport                                                                                                                                | Meta Tag for Responsive Webpages                                                                                                                                                                | Grid View – pg. 48<br>grid-view.html<br>Background Size – pg. 49<br>N/A<br>Images – pg. 50<br>image.html<br>Viewport – pg. 51<br>about.html                      |
| <b>Lesson 3</b><br>Video time - 00:11:41<br>Exercise Lab time - 00:24:00<br>Workbook time - 00:40:00 | <b>Responsive Design Part 2</b><br>Responsive Width<br>Media Queries<br><b>Flexible Responsive Layouts</b><br>Flex Container<br>Flex Items | 4.2.6 Responsive width<br>4.2.7 Media queries<br>4.3 Construct flexible, responsive layouts by using CSS flexbox<br>4.3.1 Flex container (flex-direction, flex-flow, flex-wrap)<br>4.3.2 Flex items (flex-basis, flex-grow, flex-shrink, order, flex)                          | Understanding Responsive Width<br>Understanding Media Queries<br>Flex-Wrap<br>Flex-Grow<br>Flex-Shrink<br>Order Property                                                                        | Responsive Width and Media QueriesE – pg. 54<br>mediaquery.html<br>CSS Flexbox – pg. 55<br>flexbox.html<br>CSS Flex Items – pg. 56<br>flexbox-items.html         |
| <b>Lesson 4</b><br>Video time - 00:08:33<br>Exercise Lab time - 00:16:00<br>Workbook time - 00:45:00 | <b>Grid-Based Layouts</b><br>Containers<br>Items<br>Templates<br>Gaps and Lines                                                            | 4.4 Construct grid-based layouts by using CSS grid<br>4.4.1 Container<br>4.4.2 Items<br>4.4.3 Templates<br>4.4.4 Gap/Lines                                                                                                                                                     | Grid-Template-Columns Property<br>Grid-Column Property<br>Grid-Area Property<br>Using Developer Tools with Grids                                                                                | Grid Container – pg. 59<br>grids1.html<br>Grid Items – pg. 60<br>blog.html<br>Grid Templates – pg. 61<br>blog1.html<br>Grid Gaps and Lines – pg. 62<br>blog.html |

# Domain 5 Lesson Plan

| Domain 5 - JavaScript Coding [approximately 6.5 hours of videos, labs, and projects]                 |                                                                                                                    |                                                                                                                                                                                                  |                                                                                                                    |                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lesson                                                                                               | Lesson Topic and Subtopics                                                                                         | Objectives                                                                                                                                                                                       | Exercise Labs                                                                                                      | Workbook Projects and Files                                                                                                                                                                       |
| <b>Pre-Assessment</b><br>Assessment time - 00:40:00                                                  | JavaScript Coding: Pre-Assessment                                                                                  |                                                                                                                                                                                                  |                                                                                                                    |                                                                                                                                                                                                   |
| <b>Lesson 1</b><br>Video time - 00:13:04<br>Exercise Lab time - 00:08:00<br>Workbook time - 00:45:00 | <b>Custom Classes</b><br>Instantiation<br>Properties<br>Methods<br>Inheritance                                     | 5.1 Create and use custom classes<br>5.1.1 Instantiation<br>5.1.2 Properties<br>5.1.3 Methods<br>5.1.4 Inheritance                                                                               | Understanding Properties<br>Understanding Loops                                                                    | Instantiation – pg. 65<br>index.html; classes.js<br>Properties – pg. 66<br>index.html; classes.js<br>Methods – pg. 67<br>index.html; classes.js<br>Inheritance – pg. 68<br>index.html; classes.js |
| <b>Lesson 2</b><br>Video time - 00:12:14<br>Exercise Lab time - 00:08:00<br>Workbook time - 00:20:00 | <b>Data Access Through JavaScript Part 1</b><br>Send and Receive Data<br>XML                                       | 5.2 Perform data access by using JavaScript<br>5.2.1 Send and receive data<br>5.2.2 XML                                                                                                          | XML Objects                                                                                                        | Sending and Receiving Data – pg. 71<br>ajax.html<br>XML – pg. 72<br>xmlData1.html                                                                                                                 |
| <b>Lesson 3</b><br>Video time - 00:10:22<br>Exercise Lab time - 00:00:00<br>Workbook time - 00:30:00 | <b>Data Access Through JavaScript Part 2</b><br>JSON<br>Transmit and Parse Complex Objects<br>Load and Save Files  | 5.2.3 JSON<br>5.2.4 Transmit and parse complex objects<br>5.2.5 Load and save files                                                                                                              | N/A                                                                                                                | JSON – pg. 75<br>jsonData1.html<br>Complex Objects – pg. 76<br>jsonData2.html<br>Load and Save Files – pg. 77<br>fileReader.html                                                                  |
| <b>Lesson 4</b><br>Video time - 00:13:23<br>Exercise Lab time - 00:16:00<br>Workbook time - 00:25:00 | <b>Event Listeners and Handlers</b><br>Gesture Events<br>Multiple Events<br>Event Object<br>Bubbling vs. Cascading | 5.3 Construct code that responds to events by using event listeners and handlers<br>5.3.1 Gesture events<br>5.3.2 Handling multiple events<br>5.3.3 Event object<br>5.3.4 Bubbling vs. cascading | Event Listeners<br>Understanding Multiple Events<br>UseCapture Attribute<br>Understanding the UseCapture Attribute | Gesture Events – pg. 80<br>schedule.html; events1.js<br>Multiple Events – pg. 81<br>schedule.html; events2.js<br>Event Object – pg. 82<br>index.html; coords.js                                   |
| <b>Lesson 5</b><br>Video time - 00:14:17<br>Exercise Lab time - 00:08:00<br>Workbook time - 00:30:00 | <b>JavaScript APIs</b><br>Google Charts<br>jQuery<br>Geolocation                                                   | 5.4 Construct code that uses JavaScript APIs<br>5.4.1 Google Charts<br>5.4.2 jQuery<br>5.4.3 Geolocation                                                                                         | Using jQuery                                                                                                       | Google Charts – pg. 85<br>about.html; chart.js<br>jQuery – pg. 86<br>jquery.html<br>Geolocation – pg. 87<br>contact.html; location.js                                                             |
| <b>Lesson 6</b><br>Video time - 00:05:34<br>Exercise Lab time - 00:00:00                             | <b>Application States</b><br>Session State vs. App State<br>Local vs. Session Storage                              | 5.5 Manage the state of an application<br>5.5.1 Session state vs. app state<br>5.5.2 Where to store state (local vs. session storage)                                                            | N/A                                                                                                                | Local and Session Storage – pg. 90<br>contact.html; storage.js                                                                                                                                    |

## Domain 5 - JavaScript Coding [approximately 6.5 hours of videos, labs, and projects]

| Lesson                                               | Lesson Topic and Subtopics            | Objectives | Exercise Labs | Workbook Projects and Files |
|------------------------------------------------------|---------------------------------------|------------|---------------|-----------------------------|
| Workbook time - 00:20:00                             |                                       |            |               |                             |
| <b>Post-Assessment</b><br>Assessment time - 01:30:00 | JavaScript Coding:<br>Post-Assessment |            |               |                             |